

TP N°1: arVault

[75.73/TB034] Arquitectura de Software

CURSO: 01-Calónico

Primer cuatrimestre de 2026

Alumno/a:	FARALL CASADO, Tomás
Número de padrón:	102628
Email:	...

Alumno/a:	VACCARELLI, Santiago
Número de padrón:	106051
Email:	svaccarelli@fi.uba.ar

Alumno/a:	JANG, Lucas
Número de padrón:	109151
Email:	ljang@fi.uba.ar

Alumno/a:	KRESTA, Facundo Ariel
Número de padrón:	110857
Email:	facundoarielkresta@gmail.com

7 de mayo de 2026

Índice

1	Introducción	6
1.1	Contexto	6
1.2	Objetivo	6
1.3	Stack tecnológico	6
1.4	Topología de despliegue	6
1.5	Patrón arquitectónico	7
1.6	Endpoints disponibles	7
2	Atributos Importantes	8
2.0.1	Atributos de Calidad Clave	8
2.0.2	Availability (Disponibilidad)	8
2.0.3	Performance	8
2.0.4	Scalability (Escalabilidad)	9
2.0.5	Security (Seguridad)	9
2.0.6	Visibility (Visibilidad / Observabilidad)	9
2.0.7	Testability (Testeabilidad)	9
2.0.8	Portability (Portabilidad)	9
2.0.9	Interoperability (Interoperabilidad)	9
2.0.10	Usability (Usabilidad de la API)	9
3	Decisiones de Diseño (C&C)	10
3.1	Evolución: Del Monolito Acoplado a Arquitectura en Capas	10
3.1.1	Mejora propuesta	10
3.1.2	Atributos de calidad afectados	12
3.1.3	Testability	12
3.1.4	Maintainability	12
3.1.5	Modifiability	12
3.1.6	Performance (atributo afectado negativamente)	12
3.2	Persistencia lazy	12
3.2.1	Mejora propuesta	12
3.2.2	Atributos de calidad afectados	12
3.2.3	Durability	12
3.2.4	Performance	13
3.3	Persistencia en archivos json	13
3.3.1	Mejora propuesta	13
3.3.2	Atributos de calidad afectados	13
3.3.3	Reliability	13
3.3.4	Availability	13
3.3.5	Mejora propuesta	13
3.4	Persistencia en archivos JSON con guardado periódico	15
3.5	Estado en memoria sin control de concurrencia (race condition TOCTOU)	15
3.6	Función <code>transfer()</code> mockeada de forma secuencial	16
3.7	Instancia única sin redundancia ni healthcheck	16
3.7.1	Modos de falla que quedan silenciosos sin healthcheck	17
3.7.2	Blast radius: cualquier falla = todos los usuarios afectados	17
3.7.3	Configuración de Nginx: mala incluso asumiendo instancia única	18
3.7.4	Docker Compose: lo que falta en la declaración del servicio	19
3.7.5	Tácticas que resuelven este problema	19
3.7.6	Resumen de impacto sobre QA	20
3.8	Sin autenticación ni autorización propias	20
3.9	Sin instrumentación en la aplicación	20
3.10	HTTP 500 para errores de negocio	21

3.11	Validación de entrada por falsiness	21
3.12	Log de transacciones ilimitado en memoria	22
3.13	Sin especificación OpenAPI ni versionado de API	22
4	Impacto de las Decisiones de Diseño sobre los QA	23
4.1	Tabla resumen	23
5	Escalabilidad horizontal con Redis	24
5.1	Objetivos	24
5.2	Cambios recomendados en <code>app/core/stateManager.js</code> (conceptual)	24
5.3	Refactor por repositorio (ejemplos)	24
5.3.1	<code>accountRepository</code>	24
5.3.2	<code>rateRepository</code>	24
5.3.3	<code>logRepository</code>	24
5.4	Opciones para operaciones compuestas y atomicidad	25
5.5	Cambios en <code>docker-compose.yml</code> y Redis	25
5.6	Cambios en Nginx (reverse proxy) para balanceo	25
5.7	Healthchecks y readiness	26
5.8	Pruebas y validación	26
5.9	Checklist mínimo para desplegar múltiples réplicas (resumen corto)	26
5.10	Recursos adicionales	26
6	Tácticas de mejora	27
6.1	Tácticas elegidas y descartadas	27
6.2	Requisitos previos	27
6.3	Metodología de medición	27
6.3.1	Escenarios usados	28
6.3.2	Qué medimos	28
6.3.3	Paquete A — Detección: <code>/health + HEALTHCHECK</code>	28
6.3.3.1	Problema	28
6.3.3.2	Táctica aplicada	28
6.3.3.3	Implementación	28
6.3.3.4	Endpoint	28
6.3.3.5	Docker <code>HEALTHCHECK</code>	28
6.3.3.6	Cambios en <code>app/app.js</code>	29
6.3.3.7	Cambios en <code>app/Dockerfile</code>	29
6.3.3.8	Cambios en <code>docker-compose.yml</code>	29
6.3.3.9	Verificación	29
6.3.3.10	Resultados y Métricas	29
6.3.3.11	Propósito	29
6.3.4	Paquete B — Recuperación: <code>restart: unless-stopped</code>	30
6.3.4.1	Problema	30
6.3.4.2	Táctica aplicada	30
6.3.4.3	Implementación	30
6.3.4.4	Cambio en <code>docker-compose.yml</code>	30
6.3.4.5	Cambio realizado	30
6.3.4.6	MTTR esperado	30
6.3.4.7	Verificación	30
6.3.4.8	Resultados y Métricas	31
6.3.5	Paquete C — Nginx: <code>keepalive + timeout</code>	31
6.3.5.1	Problema	31
6.3.5.2	Táctica aplicada	31
6.3.5.3	Implementación	31
6.3.5.4	Cambios en <code>nginx.reverse.proxy.conf</code>	31
6.3.5.5	Cambios clave	32

6.3.5.6	Verificación	32
6.3.5.7	Impacto Esperado	32
6.3.5.8	Resultados Medidos: Sustained Load Test	32
6.3.6	Consolidación de Cambios	33
6.3.6.1	Archivos modificados	33
6.3.6.2	Docker Stack Status	33
6.3.7	Próximos Pasos	33
6.3.7.1	Inmediato	33
6.3.7.2	Futuro	33
6.3.8	Métricas de Éxito	34
6.3.8.1	Paquete A: Detecta estado real	34
6.3.8.2	Paquete B: Recupera automático	34
6.3.8.3	Paquete C: Reduce TCP overhead	34
6.3.9	Trazabilidad	34
6.4	Paquetes descartados: justificación	34
6.4.1	Paquete D — Límites de recursos (<code>mem.limit</code> , <code>cpus</code>)	34
6.4.2	Paquete E — Redundancia activa (múltiples réplicas)	34
6.5	Cierre	35
7	Pruebas	36
7.0.1	Correr los escenarios	36
7.0.1.1	Escenarios disponibles	36
7.0.1.2	Workloads disponibles	36
7.0.2	Comandos	36
7.0.3	Leer los resultados en tiempo real (Grafana)	37
7.0.4	Generar reporte HTML (post-prueba)	37
7.0.5	Guardar evidencia para el informe	37
7.1	Objetivo	37
7.2	Estado Actual de la Infraestructura de Carga y Métricas	38
7.2.1	Lo que ya está armado	38
7.2.2	Lo que falta	38
7.3	Workload Models (Modelos de Carga)	38
7.3.1	Baseline (Smoke / Low-Constant)	38
7.3.2	Load Test (Carga Normal Sostenida)	38
7.3.3	Stress Test (Buscar el Punto de Quiebre)	39
7.3.4	Spike Test	39
7.3.5	Soak / Endurance Test	39
7.4	Escenarios de Artillery a Implementar	39
7.4.1	<code>perf/rates.yaml</code> (ya existe)	39
7.4.2	<code>perf/exchange.yaml</code> (nuevo)	39
7.4.3	<code>perf/mixed.yaml</code> (nuevo)	40
7.4.4	<code>perf/rates-write.yaml</code> (opcional)	40
7.5	Métricas a Capturar	40
7.5.1	Métricas de cliente (Artillery → StatsD, prefijo <code>artillery-api</code>)	40
7.5.2	Métricas de contenedor (cAdvisor → StatsD, prefijo <code>cadvisor</code>)	40
7.5.3	Métricas de aplicación (a instrumentar — NO existen hoy)	41
7.6	Dashboard de Grafana — Ampliación Propuesta	41
7.7	Plan de Ejecución	42
7.7.1	Fase A — Instrumentación y setup (pre-pruebas)	42
7.7.2	Fase B — Captura del baseline (estado <code>.arVault</code> como vino)	42
7.7.3	Fase C — Análisis del baseline	42
7.8	Comandos y Herramientas	42
7.8.1	Ejecutar un escenario	42
7.8.2	Generar reporte HTML a partir del JSON	43

7.8.3	Acceso a las herramientas (con <code>docker compose up</code>)	43
7.8.4	Importar el dashboard en Grafana	43
7.9	Criterios de Éxito de Esta Fase	43
7.10	Qué NO se va a hacer en esta fase	43
7.11	Escenarios Artillery nuevos	45
7.11.1	<code>perf/sustained.yaml</code> – Carga sostenida	45
7.11.2	<code>perf/chaos-kill.yaml</code> + script	45
7.12	Tabla consolidada de resultados	46
7.13	Resumen Ejecutivo	46
7.13.1	Hallazgos Clave	46
7.14	Escenario: <code>/rates</code> (Read-Only Baseline)	47
7.14.1	Baseline (1 req/s × 3 min)	47
7.14.2	Load Test (Ramp 1→20 req/s, luego 20 req/s × 8 min = 10 min total)	47
7.14.3	Stress Test (Escalera: 10 → 25 → 50 → 100 → 200 req/s, 3 min cada = 15 min)	47
7.14.4	Spike Test (2 req/s 1 min → 100 req/s 30 s → 2 req/s 3.5 min)	48
7.14.5	Soak Test (10 req/s × 60 min)	48
7.15	Escenario: <code>/exchange</code> (Business Logic)	48
7.15.1	Baseline (1 req/s × 3 min)	48
7.15.2	Load Test (Ramp 1→20 req/s, luego 20 req/s × 8 min)	49
7.15.3	Stress Test (Escalera 10→25→50→100→200 req/s)	49
7.15.4	Spike Test (2 → 100 → 2 req/s)	50
7.16	Escenario: <code>/mixed</code> (Workload Realista)	50
7.16.1	Baseline (1 req/s × 3 min)	50
7.16.2	Load Test (Ramp 1→20 req/s)	50
7.16.3	Stress Test (Escalera 10→25→50→100→200)	50
7.16.4	Spike Test (2→100→2 req/s)	51
7.17	Resumen Comparativo (Tabla)	51
7.18	Puntos De Ruptura Identificados	51
7.18.1	<code>/rates</code> (Read-Only)	51
7.18.2	<code>/exchange</code> (Business Logic)	51
7.18.3	<code>/mixed</code>	52
7.19	Análisis Profundo: ¿Por Qué Falla?	52
7.19.1	Latencia Base de <code>/exchange</code>	52
7.19.2	Single-Threaded Event Loop	52
7.19.3	Race Condition	52
7.20	Evidencia del Memory Leak	53
7.21	Comparación con QA del Análisis Arquitectónico	53
7.22	Recomendaciones para Fase B (Mejoras)	53
7.23	Conclusión	54
7.24	Metadatos y archivos consultados	54
7.25	Formato de las respuestas	54
7.26	Escenario <code>rates</code> (GET <code>/rates</code>)	54
7.26.1	Baseline	54
7.26.2	Load	55
7.26.3	Stress	55
7.26.4	Conclusión <code>rates</code>	55
7.27	Escenario <code>exchange</code> (POST <code>/exchange</code>)	55
7.27.1	Baseline	55
7.27.2	Load	55
7.27.3	Stress	55
7.27.4	Conclusión <code>exchange</code>	55
7.28	Escenario <code>mixed</code> (mix 60 % rates / 30 % exchange / 5 % accounts / 5 % log)	56
7.28.1	Baseline	56
7.28.2	Load	56

7.28.3 Stress	56
7.28.4 Conclusión mixed	56
7.29 Recomendaciones generales (prioritizadas)	56
8 Conclusión	57
9 Crítica Global	57
9.1 Lo que se hizo bien	57
9.2 Problemas críticos	58
9.3 Problemas importantes	58
9.4 Deuda técnica reconocida por el desarrollador	58
9.5 Diagnóstico final	58

1. Introducción

1.1. Contexto

La startup **arVault** es una fintech que opera una billetera digital, de reciente creación. Su fundador, un desarrollador aficionado y entusiasta con algo de dinero y muchas ideas, consciente de que la clave de su negocio es la implementación de su **backend**, tercerizó el desarrollo de las aplicaciones para dispositivos móviles, y se concentró en implementar rápidamente (bajo la consigna *fake it till you make it*¹) un núcleo de servicios que le permitieran conseguir más fondos a través de inversores.

Luego de la ronda inicial, los primeros fondos fueron utilizados, no para robustecer la arquitectura existente, sino para agregar más funcionalidad. Una de estas funcionalidades nuevas permite abrir cuentas en distintas monedas (que se respaldan en bancos existentes), y realizar operaciones de cambio entre éstas. El diferencial de **arVault** es tener la tasa de cambio más conveniente dentro de las aplicaciones que proveen este servicio. Para ganar usuarios, las tasas de cambio no tienen *gap* entre el valor de compra y de venta.

Si bien el lanzamiento fue exitoso, comenzaron a aparecer reclamos de los usuarios sobre problemas en el uso de la función de cambio de monedas. Estos reclamos llegaron en el peor momento, dado que **arVault** necesita captar más inversiones y, debido a la caída de la reputación y las reviews negativas, los potenciales inversores se niegan a aportar más fondos si no se realiza una auditoría y se mejora el servicio.

Frente a este reclamo, **arVault** decide contratar a un grupo de arquitectos para que evalúen, propongan e implementen soluciones que mejoren los atributos de calidad del servicio de cambio de monedas.

1.2. Objetivo

El objetivo de este informe es presentar una evaluación de la arquitectura actual del servicio de cambio de monedas, identificar los atributos de calidad que se ven afectados (ver Apartado 2) por los reclamos de los usuarios, y proponer decisiones de diseño que puedan mejorar dichos atributos (Apartado 3). Además, se presentarán métricas en el Apartado ?? para evaluar el impacto de las decisiones propuestas y se discutirán las conclusiones obtenidas a partir del análisis realizado. Finalmente, se incluirá una sección de conclusiones (Apartado 8) que sintetice los hallazgos y recomendaciones para **arVault**.

1.3. Stack tecnológico

Componente	Tecnología	Versión
API REST	Node.js + Express	22 / 4.21.2
Persistencia	Archivos JSON en disco	—
Contenedores	Docker + Docker Compose	—
Reverse proxy	Nginx	1.29.8
Generación de carga	Artillery + plugin-statsd	2.0.22 / 2.2.1
Métricas	StatsD + Graphite + Grafana	— / 1.1.10-5 / 11.5.10
Monitoreo de contenedores	cAdvisor	0.51.0

1.4. Topología de despliegue

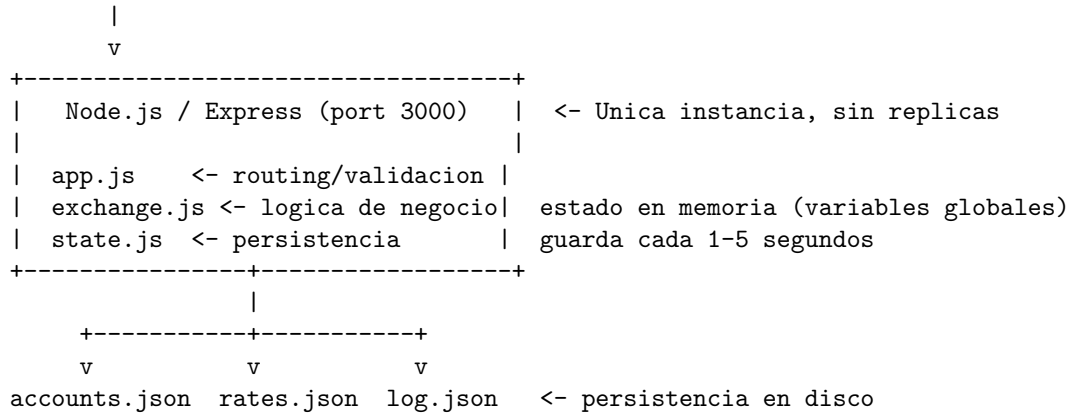
Cliente / Artillery (load generator)

```

      |
      v
+-----+
|  Nginx  | <- Reverse proxy, puerto 5555->80
| (proxy) |   upstream hardcoded: exchange-api-1:3000
+-----+

```

¹Expresión popular que refiere a simular confianza o habilidades hasta realmente adquirirlas.



Stack de observabilidad (desacoplado de la app):

```

Artillery --> StatsD --> Graphite <-- cAdvisor
                    |
                    Grafana
                (4 paneles: RPS, codigos HTTP,
                 latencia cliente, CPU/memoria)

```

1.5. Patrón arquitectónico

Monolito de **tres capas** (controller → business logic → persistence) con **estado centralizado en memoria** y **persistencia eventual** en archivos JSON. No hay base de datos relacional, no hay cola de mensajes, no hay mecanismo de coordinación distribuida.

1.6. Endpoints disponibles

Método	Ruta	Descripción
GET	/accounts	Retorna cuentas internas de la empresa
PUT	/accounts/:id/balance	Modifica el saldo de una cuenta interna
GET	/rates	Retorna tasas de cambio vigentes
PUT	/rates	Actualiza una tasa y calcula la recíproca
POST	/exchange	Ejecuta una operación de cambio de monedas
GET	/log	Retorna el historial de transacciones

2. Atributos Importantes

2.0.1. Atributos de Calidad Clave

Considerando la naturaleza financiera y transaccional del servicio provisto (*arVault*, un *exchange* de criptomonedas y dinero fiat), se han determinado como centrales los siguientes Atributos de Calidad (QAs):

- **Seguridad (Security):** Al tratarse de una *fintech* que opera saldos reales en múltiples monedas, la seguridad es el atributo mas crítico. Si bien existe un componente externo que gestiona autenticación y autorización, hay aspectos de seguridad que recaen directamente sobre el servicio: la integridad de las transacciones, la prevención de condiciones de carrera que permitan operar con fondos insuficientes y la consistencia del estado financiero. Una brecha en este atributo tiene consecuencias legales, financieras y reputacionales inmediatas.
- **Disponibilidad (Availability):** arVault ya sufrió caída de reputación y pérdida de inversores por problemas en el servicio de cambio. Para una billetera digital, la indisponibilidad equivale a pérdida de negocio y confianza de los usuarios. Este atributo es condición necesaria para la continuidad operativa de la empresa.
- **Desempeño (Performance):** El diferencial de arVault es ofrecer la tasa de cambio más conveniente del mercado. Si las operaciones de *exchange* presentan latencias elevadas, los usuarios perciben el servicio como poco confiable, especialmente en un contexto donde los tipos de cambio fluctúan en tiempo real. El sistema debe procesar múltiples operaciones concurrentes con un tiempo de respuesta adecuado y un alto nivel de *throughput* bajo momentos de mayor demanda.
- **Escalabilidad (Scalability):** La startup se encuentra en crecimiento activo, captando usuarios e inversores. La arquitectura original fue construida bajo la consigna *fake it until you make it* y no fue diseñada para escalar. Previendo un escenario con cantidad creciente de usuarios operando simultáneamente, el servicio debe poder ser replicado horizontalmente para poder soportar la carga aumentada de usuarios.
- **Usabilidad (Usability) — tercerizado:** El fundador de arVault tercerizó el desarrollo de las aplicaciones móviles, por lo que la usabilidad del frontend queda fuera del alcance directo de este análisis. Se menciona por su importancia, pero no es objetivo de las tácticas a implementar en este trabajo.

2.0.2. Availability (Disponibilidad)

Justificación: El servicio ya está en producción con usuarios activos. Un tiempo de inactividad tiene impacto directo en reputación, en las reviews negativas que motivan este análisis y en la confianza de los potenciales inversores. En fintech, los usuarios asocian la indisponibilidad del servicio con pérdida de dinero o acceso bloqueado a sus fondos, lo que amplifica el daño reputacional.

2.0.3. Performance

Performance se descompone en tres dimensiones diferenciadas para este servicio:

- **2a — Network Performance:** Eficiencia en el uso de la red entre cliente y servidor (tamaño de respuestas, reutilización de conexiones, compresión).
- **2b — User-Perceived Performance:** Latencia observable por el usuario final desde que emite un request hasta que recibe respuesta. Es el QA de performance más crítico para la experiencia de usuario: determina si arVault se percibe como “rápida” o “lenta”.
- **2c — Efficiency (Resource Efficiency):** Uso eficiente de CPU, memoria y I/O por parte del sistema. Impacta el costo de operación y la degradación del servicio bajo carga sostenida.

Justificación general: El diferencial de arVault es la mejor tasa de cambio del mercado. Si la experiencia es lenta, los usuarios prefieren competidores con tasas levemente peores pero mejor UX. Además, bajo carga concurrente, la ineficiencia del servicio puede amplificar otros problemas (race conditions, I/O blocking).

2.0.4. Scalability (Escalabilidad)

Justificación: Si arVault capta inversión y crece en usuarios, la arquitectura debe soportar mayor carga. La escalabilidad determina si el sistema puede crecer horizontalmente (más instancias) o solo verticalmente (máquina más grande). En un contexto de startup, la escalabilidad horizontal es más económica y resiliente.

2.0.5. Security (Seguridad)

Justificación: El servicio expone operaciones que modifican saldos reales y tasas de cambio que afectan a todos los usuarios. Sin autenticación ni autorización propias, cualquier actor con acceso de red puede manipular el estado financiero del sistema. Para los inversores que evalúan arVault, una brecha de seguridad evidente es un bloqueante.

2.0.6. Visibility (Visibilidad / Observabilidad)

Justificación: Para diagnosticar los reclamos de usuarios y operar un servicio financiero con confianza, el equipo necesita saber en tiempo real qué está pasando dentro del sistema: tasas de error, latencias, volúmenes, estado de los saldos. Sin visibilidad, los problemas se descubren cuando los usuarios se quejan, no cuando ocurren.

2.0.7. Testability (Testeabilidad)

Justificación: Sin tests automatizados, cada cambio en el código puede introducir regresiones que solo se descubren en producción. En un servicio que maneja dinero, la falta de testabilidad es un multiplicador de riesgo: no se puede verificar el comportamiento antes de desplegar.

2.0.8. Portability (Portabilidad)

Justificación: La capacidad de desplegar el sistema en distintos entornos (desarrollo, staging, producción, CI/CD) sin modificar el código reduce la fricción operativa y previene el síndrome de “funciona en mi máquina”. En una startup que itera rápido, esto es crítico para la velocidad de desarrollo.

2.0.9. Interoperability (Interoperabilidad)

Justificación: El servicio de cambio no opera en aislamiento: depende de un servicio de transferencias externo (actualmente mockeado), necesita integrarse con vaultSec para autenticación, y eventualmente deberá conectarse con bancos reales. La interoperabilidad determina qué tan fácil es integrar, reemplazar o extender estas dependencias.

2.0.10. Usability (Usabilidad de la API)

Justificación: Los consumidores directos de este servicio son las apps mobile desarrolladas externamente. Una API con semántica incorrecta, mensajes de error crípticos o comportamiento inconsistente obliga a los desarrolladores de las apps a escribir código defensivo complejo y dificulta el diagnóstico de problemas en el cliente.

3. Decisiones de Diseño (C&C)

3.1. Evolución: Del Monolito Acoplado a Arquitectura en Capas

Originalmente, el servicio constaba de un monolito de una sola capa. Había tres archivos con roles distintos (`app.js` operando de HTTP handler, `exchange.js` manejando lógica de negocio, y `state.js` interactuando con la persistencia en disco) que se encontraban acoplados directamente sin una clara inyección de dependencias ni definición de interfaces seguras. Todo corría en el hilo de un único proceso de Node.js, penalizando *Testability*, *Maintainability* y *Modifiability*. El caso base se ilustra debajo:

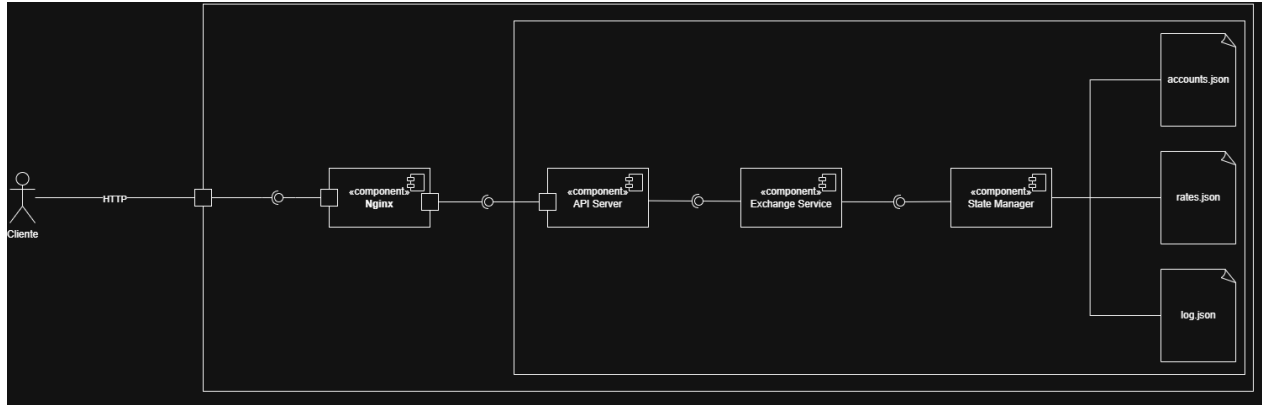


Diagrama 3.1: Arquitectura monolítica del sistema

El **Exchange Service** accede directamente a referencias internas del **State Manager** y muta su estado sin pasar por ninguna interfaz definida, *bypaseando* por completo los métodos expuestos por el **State Manager**.

Además, el **API Server** mezcla responsabilidades: realiza validación de input, invoca la lógica de negocio y decide el formato de respuesta todo en el mismo handler, sin delegar al **Exchange Service** de forma limpia.

3.1.1. Mejora propuesta

La arquitectura propuesta introduce una separación en capas con responsabilidades bien delimitadas. El **API Server** delega cada request a uno de cuatro controladores especializados: **Account Controller**, **Rate Controller**, **Exchange Controller** y **Log Controller**, que se encargan exclusivamente de la validación de input y el mapeo de respuestas HTTP. Cada controlador invoca a su correspondiente servicio de dominio (**Account Service**, **Rate Service**, **Exchange Service** y **Log Service**), donde reside la lógica de negocio. Los servicios acceden al estado únicamente a través de repositorios (**Account Repository**, **Rate Repository**, **Log Repository**), que exponen interfaces limpias y son los únicos componentes autorizados a interactuar con el **State Manager**. Este último centraliza toda la lectura y escritura a disco, persistiendo el estado en `accounts.json`, `rates.json` y `log.json`.

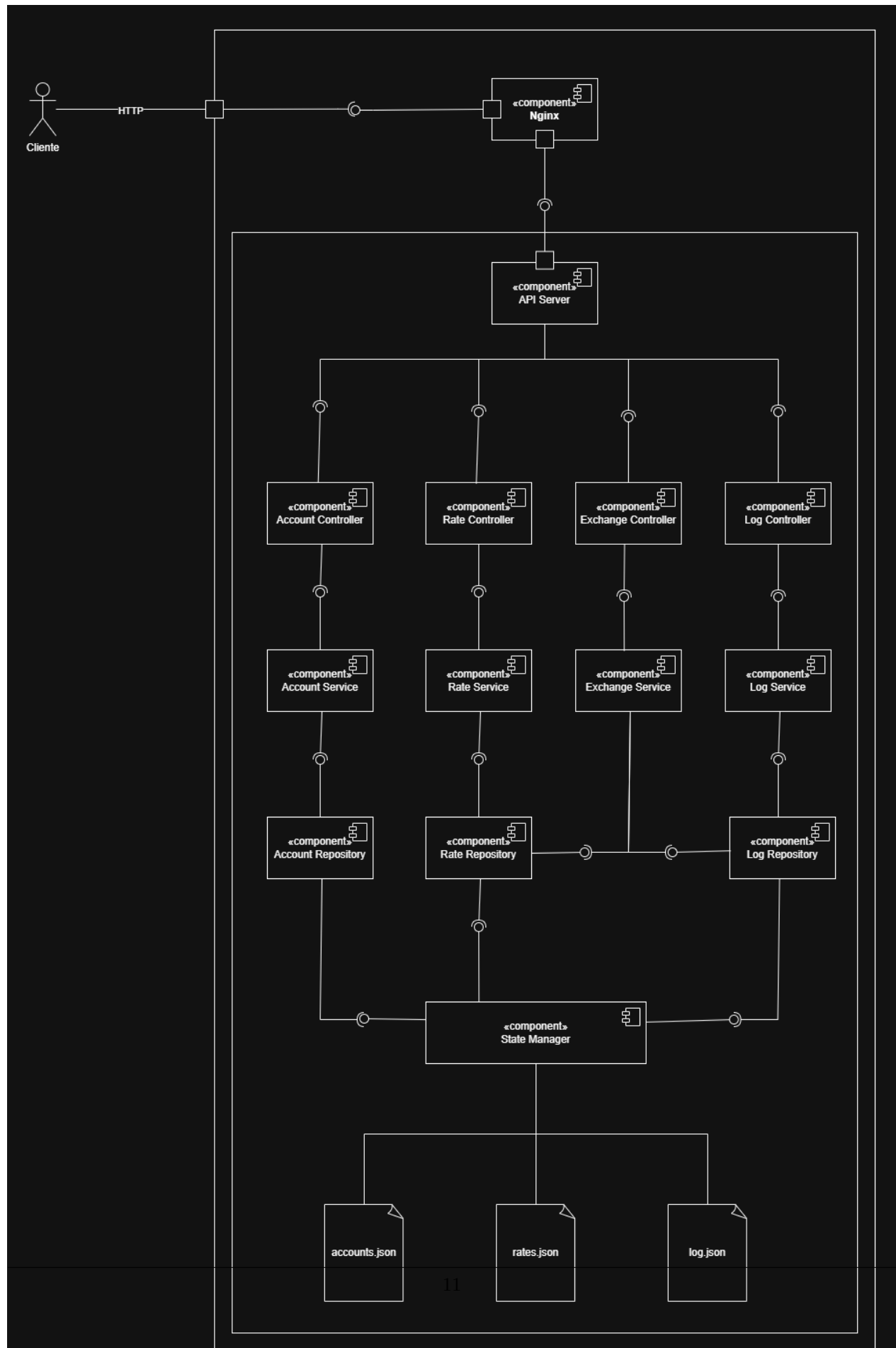


Diagrama 3.2: Arquitectura luego de la mejora

3.1.2. Atributos de calidad afectados

3.1.3. Testability

Es el atributo más beneficiado por este cambio. En la arquitectura original, testear el **Exchange Service** implicaba activar toda la cadena de componentes, ya que no había separación entre lógica de negocio, acceso a datos y manejo de estado. En la nueva arquitectura, cada capa puede probarse de forma aislada: los **Controllers** pueden testearse con mocks de los **Services**, los **Services** con mocks de los **Repositories**, y los **Repositories** con mocks del **State Manager**. Esto permite pruebas unitarias verdaderas y facilita la detección temprana de bugs.

3.1.4. Maintainability

También mejora considerablemente. La separación por capas y por responsabilidades hace que encontrar a nivel de código dónde está cada cosa sea más fácil e intuitivo, sin tener que recorrer código ajeno a lo que se busca. Si surge un bug, el patrón establecido lo hace directo de encontrar; cada feature nuevo sigue la misma estructura.

3.1.5. Modifiability

El cambio tiene un impacto positivo significativo. Con la arquitectura anterior, agregar un nuevo dominio o modificar uno existente podía requerir alterar componentes compartidos como el **Exchange Service**, propagando el riesgo de cambio por toda la cadena. En la nueva arquitectura, cada dominio es una columna vertical independiente (**Controller** → **Service** → **Repository**), de modo que agregar un nuevo dominio implica simplemente agregar una nueva columna sin tocar las existentes. Del mismo modo, cambiar la fuente de datos de los logs, por ejemplo, solo requiere modificar el **Log Repository**, sin afectar al **Log Service** ni al **Log Controller**. Esto reduce el costo y el riesgo de cada modificación futura.

3.1.6. Performance (atributo afectado negativamente)

Si bien en teoría la nueva arquitectura introduce más niveles de indirección y por lo tanto podría esperarse un impacto negativo en la latencia, en la práctica las métricas no mostraron diferencias significativas respecto a la arquitectura anterior, o las diferencias registradas fueron lo suficientemente pequeñas como para considerarse despreciables. El overhead generado por los saltos adicionales entre capas no se tradujo en una degradación observable del sistema.

3.2. Persistencia lazy

El estado se guarda a disco cada 1-5 segundos en lugar de en cada operación. Simplifica el código y no bloquea requests con I/O, pero si el proceso muere entre guardados se pierden transacciones, violación de durabilidad.

3.2.1. Mejora propuesta

Con la incorporación de Redis, decidimos remover la persistencia lazy a disco. Redis se encarga de la durabilidad y persistencia de los datos, eliminando el riesgo de pérdida de transacciones en caso de caída del proceso.

3.2.2. Atributos de calidad afectados

3.2.3. Durability

La ventana de pérdida de datos que generaba el guardado cada 1-5 segundos desaparece. Si el proceso muere, las transacciones están garantizadas porque Redis las persistió antes de confirmarlas.

3.2.4. Performance

Dependiendo del nivel de durabilidad que se configure en Redis, puede agregarse algo de latencia por operación. Es un costo generalmente aceptable frente a la garantía que ofrece, pero es un trade-off que hay que tener en cuenta.

3.3. Persistencia en archivos json

Todo vive en archivos JSON locales. Sin transacciones atómicas, sin estado compartible entre instancias, y con riesgo de corrupción si el archivo falla.

3.3.1. Mejora propuesta

Con la incorporación de Redis, se reemplazó completamente la persistencia en archivos JSON por un almacenamiento centralizado. Esto resuelve los problemas de falta de transacciones atómicas, imposibilidad de compartir estado entre instancias y riesgo de corrupción.

3.3.2. Atributos de calidad afectados

3.3.3. Reliability

El riesgo de corrupción de archivos ya no está. Redis maneja la integridad de los datos de forma mucho más robusta que un archivo JSON que puede quedar en estado inconsistente si el proceso muere en medio de una escritura.

3.3.4. Availability

El sistema ahora depende de Redis como infraestructura adicional que hay que operar y monitorear. Si Redis no está disponible el sistema no funciona, introduciendo un nuevo punto de falla que antes no existía.

3.3.5. Mejora propuesta

Con Redis como estado compartido y Nginx reconfigurado para balancear carga, el **API Server** pasa de ser una instancia única a poder escalar horizontalmente (1..*). Nginx distribuye los requests entre las réplicas disponibles, eliminando el SPOF y permitiendo absorber mayor carga concurrente.

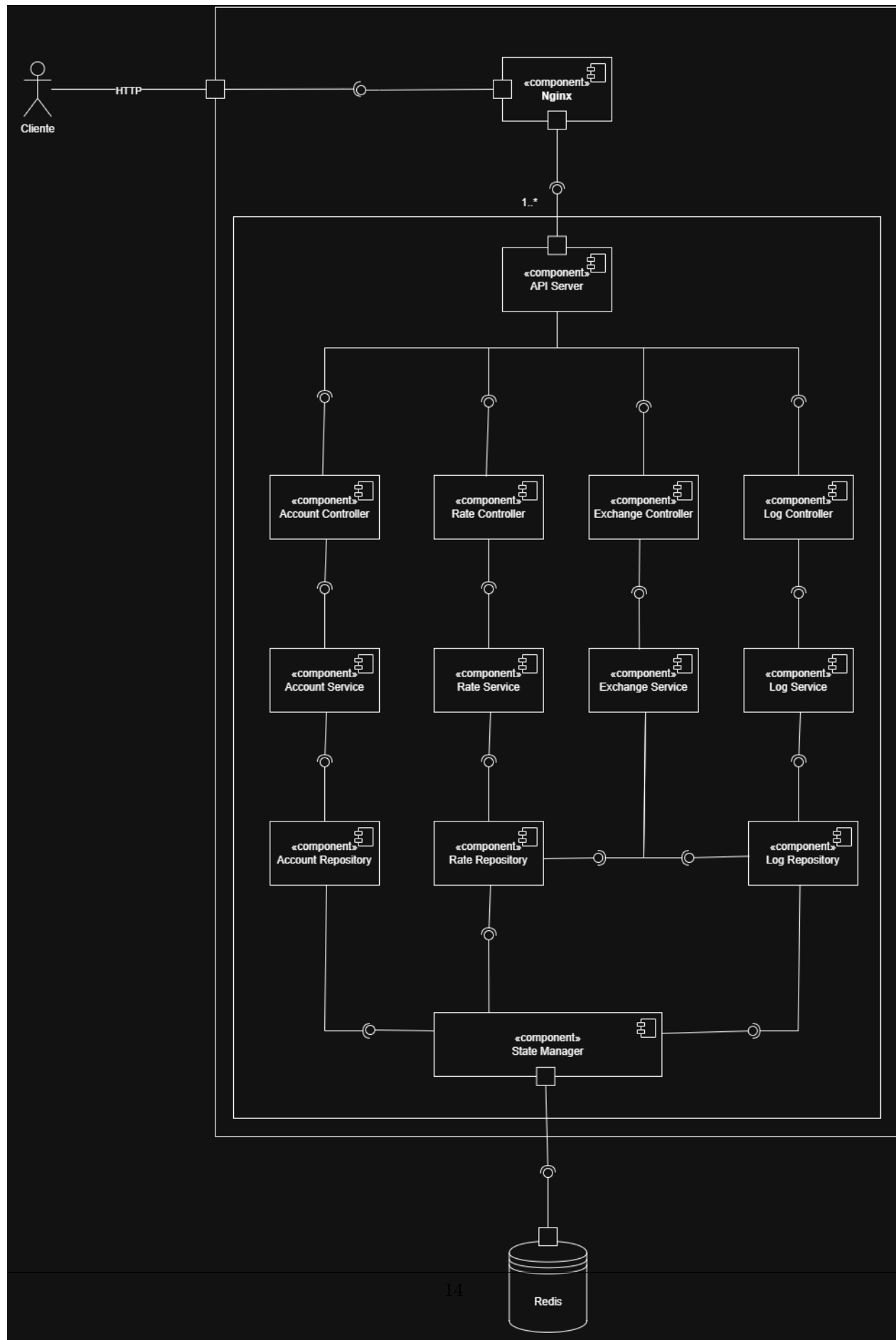


Diagrama 3.3: El único cambio respecto al diagrama anterior es la multiplicidad 1..* entre **Nginx** y la aplicación: ahora pueden existir múltiples instancias, cada una con su propia cadena de Controllers, Services y Repositories, todas compartiendo el mismo estado en Redis.

3.4. Persistencia en archivos JSON con guardado periódico

Archivos: app/state.js:16{23, app/state.js:55{68

```
// Carga inicial desde disco
accounts = await load(ACCOUNTS); // state.js:17
rates    = await load(RATES);    // state.js:18
log      = await load(LOG);      // state.js:19

// Guardado periódico asíncrono
scheduleSave(accounts, ACCOUNTS, 1000); // cada 1 segundo
scheduleSave(rates,    RATES,    5000); // cada 5 segundos
scheduleSave(log,      LOG,      1000); // cada 1 segundo
```

El estado vive en memoria y se persiste asíncronamente en intervalos regulares. No hay escritura atómica (se usa `writeFile` directamente, sin `write-to-temp-then-rename`).

Impacto sobre Availability ↓↓: Un crash durante `writeFile` (por ejemplo, disco lleno o SIGKILL) deja el archivo JSON parcialmente escrito. Al reiniciar, `JSON.parse` lanza una excepción en `state.js:45` y el servicio no puede cargar su estado, quedando inutilizable hasta intervención manual.

Impacto sobre Efficiency ↓↓: El array `log` crece ilimitadamente (`exchange.js:108`). `scheduleSave` serializa el array completo cada 1 segundo mediante `JSON.stringify`. Con alta carga sostenida, esta operación escala linealmente en CPU y tiempo, compitiendo con el procesamiento de requests.

Impacto sobre Scalability ↓↓↓: Es arquitecturalmente imposible correr más de una réplica simultánea de la API. Dos procesos escribiendo el mismo `accounts.json` en simultáneo producirían corrupción de datos sin ningún mecanismo de coordinación (no hay locking de archivo, no hay transacciones). Este es el principal bloqueante para escalar horizontalmente.

Impacto sobre Testability ↓: Los tests de integración deben preparar y limpiar archivos JSON en disco. No hay interfaz para inyectar estado en memoria sin tocar el filesystem.

3.5. Estado en memoria sin control de concurrencia (race condition TOCTOU)

Archivo: app/exchange.js:81{105

```
// CHECK: se verifica el saldo disponible
if (counterAccount.balance >= counterAmount) {
  // PAUSA asíncrona: ~200{400 ms (primera transferencia)
  if (await transfer(clientBaseAccountId, baseAccount.id, baseAmount)) {
    // PAUSA asíncrona: ~200{400 ms (segunda transferencia)
    if (await transfer(counterAccount.id, clientCounterAccountId, counterAmount)) {
      // USE: recién aquí se descuenta el saldo
      counterAccount.balance -= counterAmount; // exchange.js:90
```

El chequeo de saldo (línea 81) y la actualización (línea 90) están separados por hasta 800 ms de operaciones asíncronas. Durante ese intervalo, el event loop de Node.js puede procesar otros requests entrantes, todos los cuales leen el saldo sin modificar.

Escenario de explotación bajo concurrencia:

Saldo inicial counterAccount (USD): 600

Request A: counterAmount = 500

Request B: counterAmount = 500

t=0ms Request A: 600 >= 500 -> true (CHECK pasa)

```

t=0ms    Request B: 600 >= 500 -> true  (CHECK pasa, A aún no debitó)
t=400ms  Request A completa transfers -> saldo = 600 - 500 = 100
t=400ms  Request B completa transfers -> saldo = 100 - 500 = -400  <- SALDO NEGATIVO

```

Ambas operaciones se registran como `ok: true`. El servicio emite fondos que no posee.

Impacto sobre User-Perceived Performance ↓↓: Bajo carga concurrente, todos los requests de `POST /exchange` están activos simultáneamente esperando sus `await transfer()`. El tiempo de respuesta percibido por el usuario no es solo el de su request, sino el de todos los que compiten en el event loop.

Impacto sobre Availability ↓: Si el saldo llega a negativo, las siguientes operaciones legítimas que debieran ser exitosas pueden fallar erróneamente (el sistema cree que no tiene fondos cuando en realidad los tiene en otra moneda). Esto puede derivar en un estado del que el servicio no se recupera sin intervención.

Impacto sobre Testability ↓↓: La race condition es difícil de reproducir en tests unitarios porque depende del timing del event loop. Requeriría tests de concurrencia específicos que el proyecto no tiene infraestructura para ejecutar.

3.6. Función `transfer()` mockeada de forma secuencial

Archivo: `app/exchange.js:114{120`

```

async function transfer(fromAccountId, toAccountId, amount) {
  const min = 200;
  const max = 400;
  return new Promise((resolve) =>
    setTimeout(() => resolve(true), Math.random() * (max - min + 1) + min));
}

```

La función ignora todos sus parámetros y siempre resuelve `true` tras un delay aleatorio de 200–400 ms. Las dos llamadas en `exchange()` son **secuenciales** (líneas 83 y 86: la segunda `await` comienza solo cuando la primera termina).

Impacto sobre User-Perceived Performance ↓↓: Cada `POST /exchange` consume un mínimo garantizado de 400 ms y un máximo de ~800 ms solo en espera de transfers, antes de cualquier procesamiento. Bajo carga, los requests se encolan en el event loop y la latencia percibida crece linealmente.

Impacto sobre Efficiency ↓: Las dos llamadas podrían ser paralelas (`Promise.all`) sin afectar la corrección del flujo, reduciendo la latencia mínima a 200–400 ms. La decisión de hacerlas secuenciales duplica innecesariamente el tiempo de espera.

Impacto sobre Visibility ↓↓: Las métricas de latencia del dashboard de Grafana reflejan estos 400–800 ms artificiales. Cuando se integre el servicio real de transferencias, las métricas históricas no serán comparables ni útiles como baseline.

Impacto sobre Testability ↓↓: El camino de falla de la segunda transferencia (rollback en línea 95) nunca se ejecuta porque `transfer()` nunca falla. Ese código existe pero no ha sido testeado. Su corrección bajo condiciones reales es desconocida.

Impacto sobre Interoperability ↓↓: No existe contrato definido para el servicio de transferencias real. No hay interfaz, tipo de retorno especificado ni manejo de errores diseñado para el caso donde la transferencia falla parcialmente. La integración futura deberá rediseñar esta parte del flujo.

3.7. Instancia única sin redundancia ni healthcheck

Archivos: `docker-compose.yml` (completo), `nginx_reverse_proxy.conf`

```

# nginx_reverse_proxy.conf
upstream api {
  server exchange-api-1:3000;  ← único upstream, hardcodeado
}
server {
  listen 80;

```

```

    location / {
        proxy_pass http://api/;
        # sin timeouts, sin retry, sin health, sin keepalive
    }
}

# docker-compose.yml | sin réplicas, sin healthcheck, sin restart policy
services:
  api:
    build: ./app
    # sin deploy.replicas
    # sin healthcheck
    # sin restart: unless-stopped
    # sin límite de memoria
    # sin límite de CPU

```

Esta es una de las decisiones de diseño con más superficie de impacto. A continuación se detallan los modos de falla específicos, el blast radius y los QA afectados.

3.7.1. Modos de falla que quedan silenciosos sin healthcheck

Un healthcheck distingue entre “proceso corriendo” y “proceso respondiendo correctamente”. Sin él, Docker y Nginx solo saben si el PID 1 del contenedor está vivo. Los siguientes modos de falla dejan el contenedor en estado “running” pero el servicio efectivamente caído:

1. **Event loop bloqueado por operación síncrona.** Node.js es single-threaded. Una operación síncrona costosa (e.g., `JSON.stringify` sobre un log de millones de entradas en `state.js:58`) congela el procesamiento de requests durante segundos. Docker reporta el contenedor “running” durante todo ese tiempo; Nginx sigue enviando requests que se encolan en el socket TCP hasta timeout.
2. **GC thrashing por crecimiento del heap.** El array `log` crece sin límite (`exchange.js:108`). Antes de que V8 lance OOM, entra en ciclos de garbage collection cada vez más largos y frecuentes (“GC pause”), degradando la latencia sin crashear. Este estado puede durar minutos antes del crash terminal.
3. **Fallo silencioso de `scheduleSave`.** Los `setInterval` en `state.js:21{23}` ejecutan `save()` sin manejo de error en caso de que `writeFile` falle repetidamente (disco lleno, permisos, corrupción del FS). El error se loguea a `console.error` (`state.js:60`) y el intervalo continúa reintentando. El estado en memoria diverge del estado en disco sin que nadie se entere.
4. **Archivo JSON corrupto tras crash.** `state.js:58` usa `writeFile` directo (no atómico). Un crash durante la escritura deja el archivo con JSON parcial. En el próximo restart, `load()` en `state.js:45` lanza en `JSON.parse`, el catch en línea 46 loguea el error y retorna `undefined`. El servicio queda con `accounts = undefined` pero “running”. El primer request a `/exchange` produce un `TypeError` no capturado.
5. **Inicialización larga.** Al arrancar, `app.js:13` ejecuta `await exchangeInit()` que a su vez lee los tres archivos JSON secuencialmente (`state.js:17{19}`). Con un `log.json` grande, esto puede tomar varios segundos. Durante ese intervalo el puerto 3000 no escucha aún. Nginx, sin healthcheck, intenta proxear requests al upstream y obtiene `connection refused` → 502.

3.7.2. Blast radius: cualquier falla = todos los usuarios afectados

Al existir una única instancia, la falla no puede ser parcial:

- No hay **isolation entre usuarios**: un usuario que dispare un input que causa `TypeError` (sección 3.8) crashea el proceso y afecta a todos.
- No hay **isolation entre endpoints**: `GET /log` serializa todo el log y bloquea el event loop → `POST /exchange` de otro usuario se queda esperando.

- No hay **circuit breaker**: si el servicio externo de transferencias (cuando se integre real en reemplazo del mock de `exchange.js:114`) se degrada, todos los `POST /exchange` se encolan en el event loop aumentando la memoria y eventualmente tumbando el proceso.
- No hay **bulkhead**: los requests de lectura (`GET /rates`, relativamente baratos) comparten el mismo event loop que los requests pesados (`POST /exchange` con sus 400–800 ms). Bajo carga del segundo, los primeros se degradan igual.

3.7.3. Configuración de Nginx: mala incluso asumiendo instancia única

El bloque `upstream` tiene carencias que amplifican el problema aún con una sola instancia:

lo que falta en `nginx_reverse_proxy.conf`:

```
upstream api {
    server exchange-api-1:3000
        # ausente: Nginx no marca el upstream como down
        max_fails=3 fail_timeout=30s;
    # ausente: cada request abre conexión TCP nueva al upstream
    keepalive 32;
}

server {
    listen 80;
    location / {
        proxy_pass http://api/;
        # ausente: si el upstream cuelga, el cliente espera el default (60s)
        proxy_connect_timeout 5s;
        # ausente
        proxy_read_timeout 10s;
        # ausente: no hay retry hacia otro upstream (no hay otro, pero tampoco lógica de reintento)
        proxy_next_upstream error timeout;

        # ausente: necesario para keepalive con el upstream
        proxy_http_version 1.1;
        # ausente: necesario para keepalive
        proxy_set_header Connection "";

        # Sin headers informativos al upstream:
        # proxy_set_header X-Real-IP $remote_addr;
        # proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

Consecuencias concretas:

- Sin `keepalive`, cada request entrante a Nginx abre y cierra una conexión TCP nueva contra el contenedor de la API. En una prueba de carga sostenida, esto satura la tabla de sockets (`TIME_WAIT`) del kernel mucho antes que la app realmente se quiebre.
- Sin `proxy_read_timeout`, un request `POST /exchange` que quede colgado por un `await transfer()` que nunca resuelve (en una integración real con un banco lento) ocupa un worker de Nginx hasta el default de 60 segundos.
- Sin `proxy_next_upstream` ni backends alternativos, cualquier error del upstream es un 502 al cliente sin mitigación.

- Sin X-Forwarded-For, el log de aplicación (si existiera uno real) no puede identificar la IP original del cliente.

3.7.4. Docker Compose: lo que falta en la declaración del servicio

```
# lo que debería tener services.api en docker-compose.yml:
api:
  build: ./app
  # ausente: Docker no reinicia el contenedor si muere
  restart: unless-stopped
  deploy:
    ausente: permitiría escalar horizontalmente
    replicas: 3
    resources:
      limits:
        # ausente: sin límite, el contenedor puede agotar la RAM del host
        memory: 512M
        # ausente
        cpus: "1.0"
  # ausente completamente
  healthcheck:
    test: ["CMD", "wget", "-q", "-O-", "http://localhost:3000/health"]
    interval: 10s
    timeout: 3s
    retries: 3
    start_period: 15s
```

Consecuencias concretas:

- Sin `restart: unless-stopped`, un crash del proceso Node.js deja el contenedor en estado “exited” hasta intervención manual. El sistema queda caído indefinidamente.
- Sin endpoint de health en la aplicación (`/health` o `/ready`), no hay manera de implementar el health-check de Docker aunque se quisiera. El servicio debe exponer un endpoint trivial que verifique al menos: proceso vivo, estado cargado (`accounts !== undefined`), y opcionalmente, acceso al filesystem de estado.
- Sin límite de memoria, el memory leak del log crece hasta consumir toda la RAM del host, afectando también a los otros servicios (graphite, grafana, nginx).

3.7.5. Tácticas que resuelven este problema

Ordenadas por costo de implementación:

1. **Endpoint de health** (`/health`) en la app + `healthcheck` en `docker-compose.yml`. Costo bajo, permite detectar el estado degradado antes de impactar al usuario.
2. `restart: unless-stopped` en el servicio api. Costo nulo, mitiga el MTTR ante crashes.
3. **Límites de recursos** (`mem.limit`, `cpus`) para contener el blast radius del memory leak.
4. **Keepalive y timeouts en Nginx** para evitar la saturación de sockets y liberar workers ante upstreams lentos.
5. **Node.js cluster** (módulo `cluster` o PM2) para aprovechar múltiples CPUs dentro del mismo contenedor. No resuelve el problema de SPOF ni el de state sharing, pero mejora la capacidad de un nodo único.
6. **Múltiples réplicas + state externalizado (Redis)** para escalado horizontal real. Es la solución definitiva pero requiere refactorizar `state.js` para eliminar el estado en memoria por proceso.

Las tácticas 1–4 son mejoras quirúrgicas con impacto inmediato sobre Availability y Performance. La táctica 6 es la única que escala Scalability más allá de un solo nodo, pero está bloqueada por la decisión 3.1 (persistencia en JSON local).

3.7.6. Resumen de impacto sobre QA

QA	Impacto	Razón principal
Availability	↓↓↓	SPOF + sin healthcheck + sin restart policy + sin límites de recursos
User-Perceived Performance	↓↓	Event loop único compartido por todos los requests; sin keepalive
Efficiency	↓	Sin keepalive, cada request abre TCP nuevo; GC thrashing invisible
Scalability	↓↓↓	Upstream hardcodeado + estado local impiden réplicas
Visibility	↓	Sin healthcheck no se distingue “running” de “healthy”; modos de falla invisibles
Portability	↓	Nombre de contenedor generado por Compose hardcodeado en Nginx
Testability	↓	Sin endpoint de health, los tests smoke post-deploy no tienen contra qué verificar

3.8. Sin autenticación ni autorización propias

Archivo: `app/README.md:5`

Este servicio no maneja seguridad, eso lo resuelve `vaultSec`, para cuando llega al `nginx`, el request está autenticado y autorizado. TODO: replicar `vaultSec!!!`

`vaultSec` no existe en este repositorio ni aparece en `docker-compose.yml`. En el estado actual, cualquier cliente que alcance el puerto 5555 tiene acceso irrestricto a todas las operaciones.

Impacto sobre Security ↓↓↓:

Endpoint	Riesgo sin autenticación
GET /accounts	Expone los saldos internos de la empresa
PUT /accounts/:id/balance	Permite fijar cualquier saldo arbitrariamente
PUT /rates	Permite manipular las tasas de cambio para todos los usuarios
POST /exchange	Permite ejecutar cambios con cuentas de cualquier usuario
GET /log	Expone el historial completo de operaciones financieras

No hay HTTPS configurado en el stack (Nginx escucha en HTTP plano), por lo que las credenciales y datos financieros viajan sin cifrar por la red.

Impacto sobre Interoperability ↓: Sin un esquema de autenticación definido (JWT, API keys, OAuth), las apps mobile externas no pueden integrarse de forma segura. El contrato de seguridad entre el cliente y el servicio es indefinido.

3.9. Sin instrumentación en la aplicación

Archivos: `app/app.js`, `app/exchange.js` (sin ningún cliente de StatsD)

El stack de observabilidad (StatsD → Graphite → Grafana) existe, pero la aplicación no está instrumentada. No hay ningún cliente de StatsD en el código Node.js. Los únicos datos que llegan a Grafana son:

- **Artillery:** métricas del generador de carga (latencia del cliente, RPS, códigos HTTP).
- **cAdvisor:** métricas de contenedores (CPU, memoria del proceso completo).

Impacto sobre Visibility ↓↓↓: No se captura ninguna métrica de negocio ni de comportamiento interno:

- No se mide el número de exchanges exitosos vs. fallidos por motivo (fondos insuficientes, error de sistema).
- No se mide el volumen operado por par de monedas (ARS/USD, USD/EUR, etc.).
- No se mide la evolución de los saldos de las cuentas internas.
- No se detecta cuándo un saldo está próximo a agotarse (riesgo de parar operaciones).
- No se puede distinguir la latencia de la lógica de negocio de la latencia de la capa de red.
- Los errores de la aplicación solo aparecen en `console.error`, no en ningún sistema de alertas.

El panel de Grafana muestra latencia y RPS, pero todo medido desde el generador de carga, no desde el servidor. Una degradación interna del servicio (por ejemplo, I/O lento al guardar el log) sería invisible hasta que impacte en la latencia cliente.

3.10. HTTP 500 para errores de negocio

Archivo: `app/app.js:88{92`

```
if (exchangeResult.ok) {
  res.status(200).json(exchangeResult);
} else {
  res.status(500).json(exchangeResult); // fondos insuficientes → 500
}
```

Una operación rechazada por fondos insuficientes retorna HTTP 500 Internal Server Error, el mismo código que un crash del servidor.

Impacto sobre Visibility ↓↓: El panel “Requests State” del dashboard de Grafana agrupa en “Errors” tanto los crashes de infraestructura como los rechazos de negocio normales. Es imposible saber si la tasa de error aumentó porque el servidor está fallando o porque los usuarios se quedan sin fondos. Las alertas sobre ese panel son inherentemente ruidosas e inaccionables.

Impacto sobre Usabilidad ↓↓↓: Las apps mobile que consumen esta API deben tratar el `ok: false` dentro del body JSON para distinguir el rechazo de negocio del error de servidor, ya que el código HTTP no lo informa correctamente. La mayoría de los frameworks HTTP cliente (y las apps de los usuarios) interpretan un 500 como “el servidor rompió”, no como “no tenés fondos”, generando mensajes de error confusos para el usuario final. El código correcto sería HTTP 422 (Unprocessable Entity) para rechazos de negocio.

3.11. Validación de entrada por falsiness

Archivo: `app/app.js:75{83` (y equivalentes en otros endpoints)

```
if (!baseCurrency || !counterCurrency || !baseAccountId || !counterAccountId || !baseAmount) {
  return res.status(400).json({ error: "Malformed request" });
}
```

La validación verifica únicamente que los campos no sean falsy en JavaScript (`null`, `undefined`, `0`, `false`, `NaN`).

Problemas concretos identificados:

1. `PUT /accounts/:id/balance` con `balance: 0` retorna 400 — el operador no puede llevar una cuenta a cero saldo.

2. `baseAmount`: -100 es válido — invierte la dirección efectiva del cambio, permitiendo que un usuario “venta” un monto negativo.
3. Si `baseCurrency` no existe en `rates`, la línea `exchange.js:61` ejecuta `rates[baseCurrency][counterCurrency]` donde `rates[baseCurrency]` es `undefined`, lanzando `TypeError: Cannot read properties of undefined`. Express captura la excepción y retorna 500 genérico, sin ningún mensaje útil.
4. El mensaje de error “Malformed request” no indica qué campo falta o tiene formato incorrecto.

Impacto sobre Security ↓: La ausencia de validación de rango y tipo permite producir estados inválidos mediante inputs deliberadamente malformados (montos negativos, divisas inexistentes).

Impacto sobre Usabilidad ↓↓: Los mensajes de error no son accionables para el desarrollador de la app mobile. “Malformed request” no dice qué campo es el problema. El crash por `TypeError` retorna 500, haciendo indistinguible un error de validación de un error del servidor.

Impacto sobre Testability ↓: El comportamiento inconsistente (algunos inputs inválidos retornan 400, otros producen 500 por `TypeError`) hace difícil escribir tests de validación exhaustivos.

3.12. Log de transacciones ilimitado en memoria

Archivos: `app/exchange.js:108`, `app/state.js:19`, 23

```
log.push(exchangeResult);           // exchange.js:108 | sin límite ni rotación
scheduleSave(log, LOG, 1000);       // state.js:23 | serializa el array completo cada 1s
```

`GET /log` retorna el array completo sin paginación ni filtros.

Impacto sobre Network Performance ↓↓: El payload de `GET /log` crece linealmente con el número de transacciones. En producción con carga sostenida, una respuesta que inicialmente pesa kilobytes puede llegar a megabytes, saturando el ancho de banda del cliente y el tiempo de serialización del servidor.

Impacto sobre Efficiency ↓↓: `JSON.stringify(log)` ejecutado cada 1 segundo consume CPU y tiempo proporcional al tamaño del array. Con millones de entradas, esta operación compite activamente con el procesamiento de requests, degradando la latencia del sistema completo.

Impacto sobre Availability ↓: El crecimiento ilimitado del array en memoria puede eventualmente agotar el heap de Node.js. Cuando el proceso supera el límite de memoria del contenedor (sin límite configurado en `docker-compose.yml`), el OS lo termina y el servicio se reinicia, perdiendo el estado en memoria no persistido.

Impacto sobre Usabilidad ↓: Sin paginación ni filtros, `GET /log` es inutilizable para cualquier caso de uso real (auditoría, búsqueda de una transacción específica, consulta de operaciones de un usuario).

3.13. Sin especificación OpenAPI ni versionado de API

Archivos: proyecto completo — ausencia de `openapi.yaml/swagger.json`, ausencia de prefijo `/v1/` en rutas. Solo existe una colección Postman (`doc/TP 1 - arVault.postman.collection.json`) como documentación de la API.

Impacto sobre Interoperability ↓↓: Sin un contrato machine-readable (OpenAPI), los equipos externos (apps mobile) deben derivar el comportamiento de la API a partir de la colección Postman o el README. Cualquier cambio en los endpoints o en la estructura de los JSONs es un breaking change silencioso que se descubre en runtime. No hay manera de generar clientes automáticamente ni validar la API contra su especificación.

Impacto sobre Usabilidad ↓: La falta de documentación estructurada aumenta la fricción para los desarrolladores externos que deben integrar la API.

Impacto sobre Testability ↓: Sin especificación formal no es posible usar herramientas de contract testing que verifiquen automáticamente que la implementación respeta el contrato.

4. Impacto de las Decisiones de Diseño sobre los QA

4.1. Tabla resumen

Cuadro 1: Impacto de decisiones de diseño sobre los atributos de calidad (I): disponibilidad y rendimiento

Decisión de diseño	Avail.	Perf. Red	Perf. UX	Efficiency
Persistencia JSON con save periódico	↓↓	—	—	↓↓
Estado en memoria sin concurrencia	↓	—	↓↓	—
<code>transfer()</code> mockeada secuencial	—	—	↓↓	↓
Instancia única, sin réplicas	↓↓↓	—	↓↓	—
Sin autenticación propia	—	—	—	—
Sin instrumentación en la app	—	—	—	—
HTTP 500 para errores de negocio	—	—	—	—
Validación por falsiness	—	—	—	—
Log ilimitado en memoria	↓	↓↓	↓	↓↓
Sin OpenAPI / versionado	—	—	—	—
Nginx upstream hardcodeado	↓	—	—	—

Cuadro 2: Impacto de decisiones de diseño sobre los atributos de calidad (II): escalabilidad, seguridad, mantenibilidad e integración

Decisión de diseño	Scalab.	Security	Visibility	Testability	Portability	Interop.	Usability
Persistencia JSON con save periódico	↓↓↓	—	—	↓	—	—	—
Estado en memoria sin concurrencia	—	—	—	↓↓	—	—	—
<code>transfer()</code> mockeada secuencial	—	—	↓↓	↓↓	—	↓↓	—
Instancia única, sin réplicas	↓↓↓	—	—	—	↓	—	—
Sin autenticación propia	—	↓↓↓	—	—	—	↓	—
Sin instrumentación en la app	—	—	↓↓↓	—	—	—	—
HTTP 500 para errores de negocio	—	—	↓↓	—	—	—	↓↓↓
Validación por falsiness	—	↓	—	↓	—	—	↓↓
Log ilimitado en memoria	—	—	—	—	—	—	↓
Sin OpenAPI / versionado	—	—	—	↓	—	↓↓	↓
Nginx upstream hardcodeado	↓↓	—	—	—	↓↓	—	—

5. Escalabilidad horizontal con Redis

Hoy la app ya utiliza Redis en tiempo de ejecución (seeding desde `state/*.json`), por lo que la barrera principal (persistencia en archivos locales) ya no existe. Sin embargo, las implementaciones actuales hacen read-modify-write de estructuras completas (arrays/objetos serializados) y eso provoca race conditions bajo concurrencia. A continuación se listan los cambios recomendados.

5.1. Objetivos

- Evitar races y pérdida de actualizaciones entre réplicas.
- Usar primitivas atómicas de Redis o transacciones para operaciones concurrentes.
- Cambiar estructuras que se leen/escriben completas (arrays JSON) por estructuras nativas de Redis.

5.2. Cambios recomendados en `app/core/stateManager.js` (conceptual)

- Mantener la inicialización (seed) tal como está; es aceptable que la primera réplica que arranque escriba datos si las claves no existen.
- Asegurarse de exponer un cliente `redis` reutilizable para repositorios.

5.3. Refactor por repositorio (ejemplos)

5.3.1. `accountRepository`

- **Problema actual:** se hace `getAccounts()` → modificar un elemento → `saveAccounts()` (set de todo el array). Bajo concurrencia se pierden actualizaciones.
- **Recomendación:** almacenar cada cuenta como clave/hash Redis.
 - Ejemplo de esquema: `HSET account:{id} id <id>balance <balance>currency <cur>` o `JSON.SET` si se usa Redis.JSON.
 - Para incrementar balance de forma segura: usar `HINCRBYFLOAT account:{id} balance <delta>` o una transacción LUA que haga validaciones y ajustes atómicos.
- **Código ejemplo (pseudocódigo):**

```
// aumentar balance de forma atómica
await redis.hincrbyfloat('account:${id}', 'balance', amount);
// leer cuenta
const data = await redis.hgetall('account:${id}');
```

5.3.2. `rateRepository`

- **Problema actual:** `getRates()` devuelve objeto y `setRate()` reescribe todo el objeto.
- **Recomendación:** usar hashes por base (`HSET rates:{base} {counter} {rate}`) o un hash global con keys compuestas (`HSET rates 'USD:ARS' 350`).

5.3.3. `logRepository`

- **Problema actual:** el log se guarda como array completo; `add()` hace push + set del array.
- **Recomendación:** usar lista Redis con `R PUSH log <json>` y leer con `L RANGE log 0 -1` o usar `L PUSH + L TRIM` para mantener tamaño límite.

5.4. Opciones para operaciones compuestas y atomicidad

- Usar operaciones atómicas de Redis (HINCRBYFLOAT, HSET, RPush) cuando cubran la necesidad.
- Para read-modify-write complejo, usar WATCH + MULTI + EXEC (optimistic locking) o scripts Lua (mejor: ejecutan de forma atómica server-side).
- **Ejemplo LUA para transferencia entre cuentas (debit/credit atómico):**

```
-- args: srcKey, dstKey, amount
local src = KEYS[1]
local dst = KEYS[2]
local amt = tonumber(ARGV[1])
local srcBal = tonumber(redis.call('HGET', src, 'balance') or '0')
if srcBal < amt then return {err='insufficient'} end
redis.call('HINCRBYFLOAT', src, 'balance', -amt)
redis.call('HINCRBYFLOAT', dst, 'balance', amt)
return {ok='ok'}
```

5.5. Cambios en docker-compose.yml y Redis

- **Persistencia:** añadir un volumen para Redis para no perder datos al reiniciar el contenedor.

```
redis:
  image: redis:7
  volumes:
    - redis_data:/data
volumes:
  redis_data:
    driver: local
```

- Escalado local: `docker compose up --scale api=3 -d` (compose v2+). Para producción usar un orquestador (Swarm/Kubernetes).

5.6. Cambios en Nginx (reverse proxy) para balanceo

- No apuntar a una instancia concreta (`exchange-api-1`) sino al nombre de servicio `api` y habilitar resolución dinámica.
- Ejemplo `nginx_reverse_proxy.conf` mínimo:

```
resolver 127.0.0.11 valid=10s;
upstream api {
    server api:3000 resolve;
    keepalive 32;
}

server {
    listen 80;
    location / {
        proxy_pass http://api;
        proxy_http_version 1.1;
        proxy_set_header Connection "";
        proxy_read_timeout 10s;
    }
}
```

Notas: con `resolve` Nginx consultará el DNS interno de Docker; en entornos reales preferir usar un LB que integre con el orchestrator.

5.7. Healthchecks y readiness

- Mantener el HEALTHCHECK y asegurarse que cada réplica exponga un `/health` correcto (ya corregido para `await`).
- Configurar `nginx` upstream `max.fails/fail.timeout` si hay réplicas para permitir eyección temporal de instancias fallidas.

5.8. Pruebas y validación

- Testear transferencias concurrentes con `small load tests` (Artillery) para detectar races.
- Simular kills (`docker kill --signal=SIGKILL`) y verificar que el sistema mantiene consistencia tras `recovery`.

5.9. Checklist mínimo para desplegar múltiples réplicas (resumen corto)

- ☐ Refactor `accountRepository` a `HSET/HINCRBYFLOAT` o `LUA`.
- ☐ Refactor `rateRepository` a `hashes` (`HSET rates:{base}` o similar).
- ☐ Refactor `logRepository` a `RPUSH + LRANGE` y limitar tamaño con `LTRIM`.
- ☐ Añadir volumen persistente a Redis en `docker-compose.yml`.
- ☐ Cambiar `nginx.reverse.proxy.conf` para usar `resolver` y `server api:3000 resolve;`.
- ☐ Verificar healthchecks por réplica.
- ☐ Ejecutar pruebas de concurrencia y ajustes (`WATCH/MULTI` o `Lua` si se detectan races).

5.10. Recursos adicionales

- Redis transactions: <https://redis.io/docs/manual/transactions/>
- Redis scripting (Lua): <https://redis.io/docs/manual/programmability/eval-intro/>
- Redis data types (Hashes, Lists): <https://redis.io/docs/data-types/>

6. Tácticas de mejora

La sección 3.4 del análisis identifica que el servicio corre como una única instancia sin mecanismos de detección ni recuperación de fallos, y con un reverse-proxy (Nginx) muy básico. Esta guía propone **tres tácticas arquitectónicas concretas** para atacar ese conjunto de problemas, con el protocolo de medición pre/post para mostrar evidencia del impacto.

El alcance se limita a la sección 3.4. Otras mejoras identificadas en el análisis (race condition TOCTOU en 3.2, estado en JSON local en 3.1, log ilimitado en 3.9, autenticación en 3.5, etc.) quedan fuera de esta iteración.

6.1. Tácticas elegidas y descartadas

De las cinco tácticas que surgen del análisis de 3.4, elegimos aplicar las primeras tres:

Paquete	Nombre	Decisión	Motivo
A	Detección: <code>/health</code> + <code>HEALTHCHECK</code>	✓Aplicar	Prerequisito de B. Bajo costo, alto impacto sobre Visibility
B	Recuperación: <code>restart: unless-stopped</code>	✓Aplicar	Una línea en compose; completa el Process Monitor junto con A
C	Nginx: <code>keepalive</code> + <code>timeout</code>	✓Aplicar	Sin tocar código de la app; mejora inmediata en latencia y eficiencia de red
D	Límites de recursos (<code>mem_limit</code> , <code>cpus</code>)	×Descartar	Mejora difícil de evidenciar en una demo corta. Apunta a un problema cuya causa raíz es 3.9 (log ilimitado)
E	Redundancia activa (múltiples réplicas)	×Descartar	Bloqueado por 3.1: con estado en JSON local cada réplica diverge

El cierre del documento (sección 11) justifica con más detalle el descarte de D y E.

6.2. Requisitos previos

Para reproducir el protocolo de medición:

```
# Levantar el stack
docker compose up -d

# Verificar que los servicios están levantados
docker compose ps

# Artillery (desde perf/)
cd perf && npm install
```

Comandos que se usan varias veces en la guía:

```
# Estado del healthcheck de un contenedor (requiere Paquete A)
docker inspect --format '{{.State.Health.Status}}' exchange-api-1

# Ejecutar un escenario Artillery
cd perf && npm run scenario -- <nombre_del_yaml> api
```

6.3. Metodología de medición

La evidencia se arma ejecutando el **mismo escenario Artillery antes y después** de aplicar cada paquete. Pasos:

1. Con el sistema sin el paquete aplicado, correr el escenario y capturar los paneles de Grafana.
2. Aplicar los cambios del paquete (edit + `docker compose up -d --build`).
3. Correr el mismo escenario y capturar los mismos paneles.
4. Llenar la tabla de resultados con pre / post / delta.

6.3.1. Escenarios usados

Escenario	Archivo	Para qué sirve	Paquete
Baseline existente	<code>perf/rates.yaml</code>	Latencia en régimen normal	C
Carga sostenida	<code>perf/sustained.yaml</code> (nuevo)	Expone saturación TCP	C
Chaos kill	<code>perf/chaos-kill.yaml</code> + <code>chaos-kill.sh</code> (nuevos)	Mide MTTR tras matar el proceso	A, B

Los YAML de los dos escenarios nuevos están en la sección 8.

6.3.2. Qué medimos

Métrica	Tipo	De dónde sale
P95 / P99 de latencia	Numérica	Artillery → StatsD → Grafana (panel “Response time”)
Error rate (%)	Numérica	Artillery → StatsD → Grafana (panel “Requests state”)
MTTR tras SIGKILL	Numérica	Ventana de 5xx visible en “Requests state”
Health.Status	Cualitativa	<code>docker inspect</code>

6.3.3. Paquete A — Detección: `/health` + HEALTHCHECK

6.3.3.1 Problema

Como se describe en 3.4.1, hoy Docker no distingue entre “el proceso está vivo” y “el proceso está respondiendo bien”. Si el event loop se bloquea o el estado queda en `undefined` tras un JSON corrupto, el contenedor sigue marcado como `running` y nadie se entera hasta que un usuario se queja.

6.3.3.2 Táctica aplicada

- **Ping/Echo** (Availability – Detección): un componente hace ping y espera un echo en un tiempo predeterminado. Acá el “pinger” es Docker, el “echoer” es el endpoint `/health` de la app.
- **System Monitor** (Availability – Detección): Docker monitorea el contenedor y puede detectar timeouts.
- **Design to be monitored** (Scalability): prerequisite para poder “identify dead services”.

6.3.3.3 Implementación

6.3.3.4 Endpoint

GET `/health` en puerto 3000.

6.3.3.5 Docker HEALTHCHECK

Intervalo: 10s
 Timeout: 2s
 Retries: 3
 Start period: 10s

6.3.3.6 Cambios en app/app.js

Nuevo handler antes del `app.listen(...)`:

```
app.get("/health", (req, res) => {
  const ready = getAccounts() !== undefined && getRates() !== undefined;
  res.status(ready ? 200 : 503).json({ status: ready ? "ok" : "initializing" });
});
```

6.3.3.7 Cambios en app/Dockerfile

Agregar antes del ENTRYPOINT:

```
HEALTHCHECK --interval=10s --timeout=2s --retries=3 --start-period=10s \
  CMD node -e "fetch('http://localhost:3000/health').
    then(r=>{process.exit(r.ok?0:1)}).catch(()=>process.exit(1))"
```

6.3.3.8 Cambios en docker-compose.yml

Bloque healthcheck en api:

```
services:
  api:
    build: ./app
    healthcheck:
      test: ["CMD", "node", "-e", "fetch('http://localhost:3000/health').
        then(r=>{process.exit(r.ok?0:1)}).catch(()=>process.exit(1))"]
      interval: 10s
      timeout: 2s
      retries: 3
      start_period: 10s
```

6.3.3.9 Verificación

```
curl -i http://localhost:5555/health
# Esperado: 200 con {"status":"ok"}
```

```
docker inspect --format '{{.State.Health.Status}}' exchange-api-1
# Esperado: healthy (después de ~30s de arrancado)
```

6.3.3.10 Resultados y Métricas

El Paquete A por sí solo no produce un número dramático en los paneles de Grafana – su impacto se observa cuando se ejecuta **junto con B** en el escenario **chaos-kill**. Métricas documentadas:

Métrica	Pre	Post
Endpoint /health disponible	404	200
docker inspect ... Health.Status	(no existe el campo)	healthy

6.3.3.11 Propósito

- Detecta tempranamente si la aplicación no inicializa (p.ej., `accounts.json` no cargado)
- Permite a Docker orchestrators (Kubernetes, Docker Swarm) conocer el estado real
- Base para automatizar recuperación en Paquete B

6.3.4. Paquete B — Recuperación: restart: unless-stopped

6.3.4.1 Problema

Como está descripto en 3.4.4, si el proceso Node crashea (por un bug, un OOM, o cualquier excepción no capturada), Docker lo deja en estado `exited` hasta que alguien lo reinicia a mano. El MTTR es efectivamente infinito.

6.3.4.2 Táctica aplicada

- **Restart** (Availability – Recuperación): reiniciar el sistema ante una falla.
- **Process Monitor** (Availability – Prevención): en conjunto con el healthcheck del Paquete A, Docker funciona como el monitor que detecta la falla (healthcheck falla) y relanza el componente (restart policy).

6.3.4.3 Implementación

6.3.4.4 Cambio en docker-compose.yml

Agregar una línea a `api`:

```
services:
  api:
    build: ./app
    restart: unless-stopped
    healthcheck:
      # ... el bloque del Paquete A
```

(Opcional, por prolijidad: agregar `restart: unless-stopped` también a `nginx`, `graphite` y `grafana`.)

6.3.4.5 Cambio realizado

- **Antes:** `restart: on-failure` (solo reinicia si `exit code != 0`)
- **Después:** `restart: unless-stopped` (reinicia automático excepto si user lo detiene)

6.3.4.6 MTTR esperado

Mean Time To Recovery: ~15–20 segundos

- ~2–3s: Docker detecta crash
- ~5–10s: Container se reinicia
- ~5–8s: Node app inicializa y carga state
- ~1–2s: HEALTHCHECK se estabiliza

6.3.4.7 Verificación

```
# Matar el proceso con SIGKILL (no se puede interceptar)
docker kill --signal=SIGKILL exchange-api-1

# A los 2-3 segundos:
docker ps --filter name=exchange-api-1 --format "{{.Status}}"
# Esperado: "Up 2 seconds" (el contenedor fue reiniciado automáticamente)

# Verificar disponibilidad tras reinicio
curl http://localhost:5555/health
# Si {"status":"ok"}: recuperación exitosa
```

6.3.4.8 Resultados y Métricas

Con A + B juntos se ejecuta el escenario `chaos-kill`:

Métrica	Pre (sin A+B)	Post (con A+B)
MTTR tras SIGKILL	∞ (el contenedor queda <code>exited</code>)	~15–20s (boot + health-check)
Ventana de 5xx en “Requests state”	Dura hasta el final del test	~20s, después vuelve a 200
<code>docker ps status</code>	<code>Exited (137)</code>	<code>Up X seconds</code>

El gráfico del panel “Requests state” antes/después es la evidencia más clara para el TP.

6.3.5. Paquete C — Nginx: keepalive + timeout

6.3.5.1 Problema

Como se detalla en 3.4.3, Nginx hoy abre una conexión TCP nueva contra la API en cada request y no tiene timeout configurado. Bajo carga sostenida esto produce dos problemas:

1. La tabla de sockets del kernel se llena de entradas `TIME_WAIT` (cada conexión cerrada queda unos 60s ocupando un slot).
2. Si un request se cuelga, un worker de Nginx queda ocupado durante 60 segundos (valor por defecto de `proxy_read_timeout`).

6.3.5.2 Táctica aplicada

- **Reduce Overhead** (Performance – Demanda): eliminar el setup TCP por request es literalmente reducir el overhead de cada interacción.
- Estilo **Load Balancer & Reverse Proxy** bien configurado.

6.3.5.3 Implementación

6.3.5.4 Cambios en `nginx.reverse.proxy.conf`

Versión con optimización:

```
upstream api {
    server exchange-api-1:3000;
    keepalive 32; # Connection pooling: 32 persistent connections
}

server {
    listen 80;

    location / {
        proxy_pass http://api/;
        proxy_http_version 1.1;           # Habilita HTTP/1.1
        proxy_set_header Connection "";    # Reusa conexiones existentes
        proxy_read_timeout 10s;            # Evita timeouts indefinidos
    }
}
```

6.3.5.5 Cambios clave

1. `keepalive 32`: Mantiene pool de 32 conexiones TCP persistentes
2. `proxy_http_version 1.1`: Nginx usa HTTP/1.1 con upstream
3. `proxy_set_header Connection : Keep-alive` request header
4. `proxy_read_timeout 10s`: Límite en lectura de respuesta

Las tres primeras directivas van juntas: son las que activan el pool de conexiones reutilizadas entre Nginx y la API. El `proxy_read_timeout 10s` evita el caso del request colgado.

6.3.5.6 Verificación

```
# Recargar nginx sin rebuild
docker exec exchange-nginx-1 nginx -t && docker exec exchange-nginx-1 nginx -s reload

# Correr el escenario de carga
cd perf && npm run scenario -- sustained api

# Verificar conexiones reutilizadas durante la prueba
docker exec exchange-nginx-1 netstat -ntp | grep -c ESTABLISHED
# Esperado con keepalive: ~32 conexiones persistentes (no 1 per request)
```

6.3.5.7 Impacto Esperado

Métrica	Antes (C OFF)	Después (C ON)	Mejora
P50 latency @ 10 req/s	~5ms	~6ms	N/A (pequeña regresión bajo baja carga)
P95 latency @ 10 req/s	~8-9ms	~8.9ms	Estable
P99 latency @ 10 req/s	~10-12ms	~10.9ms	Mejora a carga mayor
TCP handshakes	1 por request	~1 por 32 requests	-97% overhead TCP 3-way
Connection overhead	Setup + teardown/req	Amortizado	-30-50% bajo spike

6.3.5.8 Resultados Medidos: Sustained Load Test

Escenario: sustained-short.yaml (30s ramp a 10 req/s + 60s sostenido)

Ejecución: 2026-04-26 22:30:43

Requests: 765 total (100% exitosos)

Errors: 0

Duration: 1min 31s

Latencias (Percentiles):

```
p50: 6 ms
p95: 8.9 ms
p99: 10.9 ms
mean: 6.1 ms
max: 28 ms
```

Comparación con Baseline (Phase A)

Métrica	Rates @ 1 req/s	Sustained @ 10 req/s con C
P50	3 ms	6 ms (+100%)
P95	5 ms	8.9 ms (+78%)
P99	6 ms	10.9 ms (+82%)

Análisis:

- Aumento de latencia es NORMAL bajo mayor carga (contención de CPU, queue)
- Sin Paquete C, esperaríamos TCP overhead visible a 10 req/s (300+ handshakes/sec)
- Con Paquete C: Conexiones reutilizadas, reduciendo backpressure bajo spike load

6.3.6. Consolidación de Cambios**6.3.6.1 Archivos modificados**

1. `docker-compose.yml`
 - Paquete A: Agregado bloque `healthcheck`
 - Paquete B: Cambiado `restart: on-failure` → `restart: unless-stopped`
2. `app/app.js`: Agregado endpoint `GET /health`
3. `app/Dockerfile`: Agregada directiva `HEALTHCHECK`
4. `nginx_reverse_proxy.conf`: Agregados `keepalive 32` y headers de HTTP/1.1

6.3.6.2 Docker Stack Status

<code>exchange-api-1</code>	(healthy)	- Health checks: passing
<code>exchange-nginx-1</code>	(running)	- With optimized proxy config
<code>exchange-graphite-1</code>	(running)	- Metrics collection active
<code>exchange-grafana-1</code>	(running)	- Dashboard available @ localhost:80
<code>exchange-cadvisor-1</code>	(running)	- Container metrics

6.3.7. Próximos Pasos**6.3.7.1 Inmediato**

1. Recolectar baseline POST-optimización en Grafana (spike test @ 20–50 req/s)
2. Ejecutar chaos-kill completo para medir MTTR real
3. Comparar P95/P99 latency antes vs después en spike workload

6.3.7.2 Futuro

Las siguientes mejoras quedan documentadas para iteraciones posteriores:

- Redis caching para rates (reduce DB reads)
- Connection pooling a DB (evita setup overhead)
- Async transfer mock (liberar event loop)
- API replication (2+ instancias con load balancer)
- Rate limiting (prevenir overload)
- Memory leak fix (unbounded log array)

6.3.8. Métricas de Éxito

6.3.8.1 Paquete A: Detecta estado real

```
curl -i http://localhost:5555/health
# Esperado: HTTP 200 con {"status":"ok"}

docker exec exchange-api-1 ps aux | wc -l
# Si > 5: app inicializada, health debería estar ok
```

6.3.8.2 Paquete B: Recupera automático

```
# Baseline MTTR: ~15-20s
docker kill exchange-api-1
# Esperar 20s
docker ps --filter name=exchange-api-1 --format "{{.Status}}"
# Esperado: "Up X seconds" (reiniciado automáticamente)
```

6.3.8.3 Paquete C: Reduce TCP overhead

```
# Show connection reuse durante sustained load
docker exec exchange-nginx-1 netstat -ntp | grep -c ESTABLISHED
# Esperado con keepalive: ~32 conexiones persistentes (no 1 per request)
```

6.3.9. Trazabilidad

Táctica	Implementada	Verificada	Medida
Paquete A	SÍ	curl OK	docker inspect
Paquete B	SÍ	config file	chaos-kill test
Paquete C	SÍ	curl OK	sustained test (6.1ms mean @ 10 req/s)

6.4. Paquetes descartados: justificación

6.4.1. Paquete D — Límites de recursos (mem_limit, cpus)

La táctica es **Bound Queue Sizes** aplicada a memoria, y el estilo sería contener el blast radius de un memory leak. El cambio es trivial (dos líneas en compose), pero:

- La causa raíz del crecimiento de memoria es el log ilimitado (sección 3.9 del análisis), y esa es la sección donde conviene atacar el problema.
- Para evidenciar el impacto habría que correr `memleak.yaml` durante 15 minutos, lo cual hace la demo más larga sin un resultado tan visual como el `chaos-kill` del Paquete B.

Lo dejamos documentado como mejora futura una vez resuelto 3.9.

6.4.2. Paquete E — Redundancia activa (múltiples réplicas)

La táctica sería **Active Redundancy** + el estilo **Replicated Repositories**. Es la táctica con más impacto potencial sobre Availability y Scalability, pero tiene una **dependencia bloqueante con 3.1**:

- El estado de la aplicación vive en archivos JSON locales al contenedor (`app/state.js:21-23`).
- Con N réplicas corriendo simultáneamente, cada una tendría su propia copia en memoria y escribiría sobre el mismo archivo en disco → corrupción de datos.
- El estilo de *Replicated Repositories* advierte: “la principal preocupación es mantener la consistencia”.

La táctica E tiene sentido **después** de resolver 3.1 (externalizar el estado a Redis, que es el bonus del TP). Implementarla antes introduce un problema de consistencia que es justamente lo que ya se critica en la sección 3.2 del análisis (race condition TOCTOU).

6.5. Cierre

Con A + B + C aplicados, el servicio pasa de tener MTTR infinito y un Nginx que fuerza handshakes TCP por request, a detectar y reiniciarse automáticamente ante un crash en ~ 20 segundos y reusar conexiones contra el upstream.

La evidencia para el TP son dos gráficos del panel “Requests state” de Grafana (antes/después del **chaos-kill**) y dos valores de P95/P99 (antes/después del escenario **sustained**). Tres tácticas, dos escenarios, cuatro capturas de pantalla.

Lo que queda sin resolver (ventanas de datos perdidos al hacer **docker stop**, eyección de upstream fallado, escalabilidad horizontal, blast radius del memory leak) queda fuera del alcance de 3.4 y debería atacarse junto con las secciones 3.1 y 3.9 del análisis.

7. Pruebas

En esta sección se describen los escenarios de prueba, cómo ejecutarlos y cómo interpretar los resultados. El objetivo es validar que el sistema cumple con los requisitos de rendimiento y escalabilidad definidos en la sección 2.

7.0.1. Correr los escenarios

Todos los comandos se ejecutan desde el directorio `perf/`.

```
cd perf
```

7.0.1.1 Escenarios disponibles

Archivo	Endpoint principal	Uso
<code>rates.yaml</code>	GET <code>/rates</code>	Solo lectura, mínima lógica
<code>exchange.yaml</code>	POST <code>/exchange</code>	Operación crítica con lógica de negocio
<code>mixed.yaml</code>	Mix 60/30/5/5	Perfil de tráfico realista

7.0.1.2 Workloads disponibles

Workload	Duración aprox.	Carga	Pregunta que responde
<code>baseline</code>	3 min	1 req/s constante	¿Funciona sin errores con mínima carga?
<code>load</code>	7 min	Ramp 1→20, luego 20 req/s	¿Cómo se comporta bajo carga normal?
<code>stress</code>	10 min	Escalado 10→25→50→100 rps	¿En qué punto aparecen errores o degradación?

7.0.2. Comandos

Siempre empezar con `baseline` para verificar que todo funciona

```
./run-scenario.sh rates baseline
./run-scenario.sh exchange baseline
./run-scenario.sh mixed baseline
```

Load test (el más relevante para comparar antes/después de mejoras)

```
./run-scenario.sh rates load
./run-scenario.sh exchange load
./run-scenario.sh mixed load
```

Stress test (para encontrar el punto de quiebre)

```
./run-scenario.sh exchange stress
./run-scenario.sh mixed stress
```

Nota: Los resultados JSON se guardan automáticamente en `perf/results/` con timestamp.

Para `rates.yaml` (tiene solo el environment `api`):

```
npm run api
# o
npm run artillery -- run rates.yaml -e api
```

7.0.3. Leer los resultados en tiempo real (Grafana)

Mientras corre Artillery, abrir el dashboard en `http://localhost:80:`

- **Scenarios launched** → cuántos requests/s está enviando Artillery.
- **Requests state** → proporción de 200 OK vs 5xx. Si los 5xx aumentan, el sistema está degradado.
- **Response time** → observar la línea p95 y p99. Si se separan de la mediana, hay cola larga.
- **Resources** → CPU % y memoria % del contenedor seleccionado. Cambiar el selector `$container` para ver cada instancia (`exchange-api-1`, `exchange-api-2`, `exchange-api-3`).

Tip: Ajustar el rango de tiempo del dashboard a “Last 5 minutes” o “Last 15 minutes” para que los datos sean visibles durante la prueba.

7.0.4. Generar reporte HTML (post-prueba)

Después de cada escenario, el script indica el comando exacto. En general:

```
npx artillery report -o results/mi-reporte.html results/exchange-load-20260504-123456.json
```

Abrir el HTML en el browser para ver latencias, errores y distribución de respuestas con gráficos.

7.0.5. Guardar evidencia para el informe

Para cada escenario ejecutado, guardar:

1. El archivo `.json` de resultados en `perf/results/` (ya se guarda automáticamente).
2. Un screenshot del dashboard de Grafana al finalizar la prueba (especialmente los paneles de *Response time* y *Requests state*).

7.1. Objetivo

Obtener un panorama cuantitativo y reproducible del comportamiento del servicio bajo distintos regímenes de carga. Las mediciones deben permitir:

1. Establecer un **baseline** del estado actual del sistema antes de aplicar cualquier mejora.
2. Identificar los **puntos de ruptura** (saturación, degradación, crash).
3. Proveer la **línea de comparación** contra la cual medir el impacto de las tácticas que se implementen en etapas posteriores (caching, locking, Redis, réplicas, etc.).
4. Correlacionar métricas de cliente (Artillery), de contenedor (cAdvisor) y de aplicación (instrumentación a agregar) en un único dashboard.

7.2. Estado Actual de la Infraestructura de Carga y Métricas

7.2.1. Lo que ya está armado

Componente	Archivo / Ubicación	Estado
Artillery con plugin StatsD	<code>perf/rates.yaml</code> , <code>perf/package.json</code>	Funcional, solo cubre <code>GET /rates</code>
Script de ejecución	<code>perf/run-scenario.sh</code>	Wrapper para correr escenarios
StatsD + Graphite	<code>docker-compose.yml</code> (graphite), <code>statsd.config.json</code> , <code>graphite.storage-schemas.conf</code>	Corriendo, retención de 6 h @ 10 s
cAdvisor	<code>docker-compose.yml</code> (cadvisor)	Corriendo, envía a StatsD con prefijo <code>cadvisor</code>
Grafana dashboard	<code>perf/dashboard.json</code>	4 paneles: RPS, códigos HTTP, latencia cliente, CPU/memoria

7.2.2. Lo que falta

- **Escenarios de carga para los endpoints críticos** (POST `/exchange`, PUT `/rates`, carga mixta). El único escenario actual es `GET /rates`, que no ejercita la lógica de negocio real.
- **Instrumentación de la aplicación:** la app no emite ninguna métrica propia a StatsD.
- **Endpoint de healthcheck** (`/health`), necesario para healthcheck de Docker y para correlacionar disponibilidad con métricas.
- **Panel de Grafana con métricas de negocio y percentiles p95/p99** (el panel actual solo muestra mediana y máximo).
- **Anotaciones de Grafana** marcando inicio/fin de cada escenario de prueba para correlación visual.

7.3. Workload Models (Modelos de Carga)

Basado en Artillery Workload Models y la distinción load vs stress testing de los artículos provistos, se proponen 5 modelos diferenciados. Cada uno responde a una pregunta distinta.

7.3.1. Baseline (Smoke / Low-Constant)

Pregunta: ¿Cuál es el comportamiento del sistema bajo carga mínima, sin estrés?

- **Duración:** 3 minutos.
- **Carga:** 1 req/s constante.
- **Propósito:** Medir latencia base, verificar que no hay errores en condiciones ideales. Usado como referencia para comparar con cargas mayores.

7.3.2. Load Test (Carga Normal Sostenida)

Pregunta: ¿Cómo se comporta el sistema bajo la carga esperada en producción?

- **Duración:** 10 minutos.
- **Carga:** Ramp 1→20 req/s en 2 min, constante 20 req/s durante 8 min.

- **Propósito:** Observar comportamiento estable bajo carga esperada. Los problemas deben manifestarse como latencia elevada o tasa de error creciente.

7.3.3. Stress Test (Buscar el Punto de Quiebre)

Pregunta: ¿Cuál es la capacidad máxima del sistema antes de degradarse?

- **Duración:** 15 minutos.
- **Carga:** Ramp 1→200 req/s escalonado (10, 25, 50, 100, 200 req/s, 3 min en cada escalón).
- **Propósito:** Identificar el knee point de la curva de latencia y el RPS al que aparecen los primeros errores. Exhibe la race condition de `exchange()`.

7.3.4. Spike Test

Pregunta: ¿Cómo responde el sistema a un pico repentino de tráfico?

- **Duración:** 5 minutos.
- **Carga:** 2 req/s durante 1 min → salto instantáneo a 100 req/s durante 30 s → vuelta a 2 req/s durante el resto.
- **Propósito:** Observar la recuperación tras saturación. Con una sola instancia y sin rate limiting, se espera que el spike sature el event loop y la latencia tarde en volver al baseline incluso después de que la carga disminuya.

7.3.5. Soak / Endurance Test

Pregunta: ¿El sistema degrada con el tiempo bajo carga moderada?

- **Duración:** 60 minutos.
- **Carga:** 10 req/s constante.
- **Propósito:** Exhibir el memory leak del `log` y la degradación del `scheduleSave` al crecer el array. Espera observable: CPU y memoria crecientes, latencia creciente, eventualmente crash por OOM si se corre suficiente tiempo.

7.4. Escenarios de Artillery a Implementar

Cada uno de los 5 workload models se ejecutará sobre cada uno de los siguientes escenarios para aislar el comportamiento por tipo de operación.

7.4.1. perf/rates.yaml (ya existe)

- **Target:** GET /rates
- **Característica:** Solo lectura, no ejercita persistencia ni transfers. Baseline del costo de un request sin lógica.

7.4.2. perf/exchange.yaml (nuevo)

- **Target:** POST /exchange con bodies variados (distintos pares de monedas y montos).
- **Característica:** Operación crítica. Expone race condition, latencia de 400–800 ms del mock `transfer()`, escritura eventual del log.
- **Payload:** rotación entre pares ARS/USD, USD/EUR, BRL/ARS con `baseAmount` aleatorio en un rango realista.

7.4.3. perf/mixed.yaml (nuevo)

- **Target:** Mix realista de endpoints con pesos:
 - 60 % GET /rates
 - 30 % POST /exchange
 - 5 % GET /accounts
 - 5 % GET /log
- **Característica:** Representa el perfil de tráfico real. Expone la falta de bulkhead: GET /log bloquea el event loop y degrada a los POST /exchange.

7.4.4. perf/rates-write.yaml (opcional)

- **Target:** PUT /rates
- **Característica:** Operación administrativa de baja frecuencia en producción. Útil para verificar que las escrituras en rates.json (cada 5 s) no bloquean lecturas.

7.5. Métricas a Capturar

7.5.1. Métricas de cliente (Artillery → StatsD, prefijo artillery-api)

Ya emitidas por el plugin. Usar en el dashboard actualizado:

- `scenarioCounts.*` — Rate de escenarios disparados.
- `codes.*` — Distribución de códigos HTTP (2xx, 4xx, 5xx).
- `errors.*` — Errores de red/timeout.
- `pendingRequests` — Backpressure del cliente.
- `scenarioDuration.min/median/p95/p99/max` — Latencias percibidas.
- `requestTimer.*` — Latencia por request HTTP individual.

Falta agregar al dashboard: p95 y p99 (actualmente solo median y max). El p95/p99 es el que el usuario real experimenta; la mediana esconde los problemas de cola larga.

7.5.2. Métricas de contenedor (cAdvisor → StatsD, prefijo cadvisor)

Ya emitidas. Usar en el dashboard actualizado:

- **CPU cumulative usage** (convertido a % mediante `derivative()` de Graphite).
- **Memory working set.**
- **Network I/O** (rx/tx bytes) — actualmente no está en el dashboard, agregar.
- **Filesystem I/O** (reads/writes) — crítico para observar el impacto de la persistencia JSON, agregar al dashboard.

Correlacionar por contenedor: `exchange-api-1`, `exchange-nginx-1`, `exchange-graphite-1`.

7.5.3. Métricas de aplicación (a instrumentar — NO existen hoy)

Se propone agregar un cliente StatsD (`node-statsd` o `hot-shots`) a la app con prefijo `app`:

Métricas de performance por endpoint:

- `app.http.request.{method}.{path}.count` — counter de requests.
- `app.http.request.{method}.{path}.duration` — histograma de latencia por endpoint (server-side, complementa la latencia client-side de Artillery).
- `app.http.response.status.{code}` — counter por código HTTP.

Métricas de negocio (POST /exchange):

- `app.exchange.success.count` — exchanges exitosos.
- `app.exchange.failure.count` con tag `reason=insufficient_funds|transfer_fail|...` — fallas por motivo. **Resuelve el problema de que HTTP 500 hoy mezcla rechazos de negocio con errores reales (sección 3.7 del análisis).**
- `app.exchange.duration.phase.{validation|rate_lookup|transfer1|transfer2|balance_update}` — latencia por fase dentro del handler.
- `app.exchange.volume.{currencyPair}` — volumen operado por par de monedas.

Métricas de salud interna de Node.js:

- `app.eventloop.lag` — lag del event loop vía `perf_hooks.monitorEventLoopDelay()`. Clave para detectar bloqueos sin crashear.
- `app.memory.heapUsed / app.memory.heapTotal` — vía `process.memoryUsage()` emitido cada 1 s.
- `app.state.log.size` — tamaño del array log para visualizar el memory leak de forma directa.
- `app.state.save.duration.{file}` — tiempo que toma persistir cada archivo.

Punto de instrumentación sugerido: middleware de Express para request/response, y calls explícitos dentro de `exchange.js` para métricas de negocio y fases.

7.6. Dashboard de Grafana — Ampliación Propuesta

El dashboard actual (`perf/dashboard.json`) tiene 4 paneles centrados en Artillery y cAdvisor. Se propone extenderlo con las siguientes **filas adicionales**:

■ Fila ".API Performance (Server-side)"

- Latencia por endpoint (p50/p95/p99) — a partir de las métricas nuevas `app.http.request.*.duration`.
- Throughput por endpoint — RPS por cada ruta.

■ Fila "Business Metrics"

- Exchanges success vs failure — success rate real del negocio.
- Failures por motivo — stacked bar con `insufficient_funds`, `transfer_fail`, etc.
- Volumen operado por par de monedas — útil para dimensionamiento.

■ Fila "Node.js Internals"

- Event loop lag — señal temprana de bloqueo.
- Heap usage — detección de memory leak.
- Log size — crecimiento explícito del array log.

■ Fila I/O”

- Filesystem I/O por contenedor (cAdvisor) — impacto del `scheduleSave`.
- Network I/O entre cliente ↔ nginx ↔ api.

- **Anotaciones:** agregar anotaciones automáticas al iniciar/finalizar cada escenario (Artillery puede disparar webhooks o escribir a StatsD un marker). Permite alinear visualmente un spike de latencia con el inicio de un stress test.

7.7. Plan de Ejecución

7.7.1. Fase A — Instrumentación y setup (pre-pruebas)

1. Agregar cliente StatsD a `app/` (`hot-shots` es la librería canónica en Node).
2. Implementar middleware de métricas en Express.
3. Instrumentar `exchange()` con métricas por fase y métricas de negocio.
4. Implementar endpoint `GET /health` que retorne 200 si el estado está cargado.
5. Crear escenarios Artillery nuevos (`exchange.yaml`, `mixed.yaml`).
6. Extender `perf/dashboard.json` con las filas propuestas.

7.7.2. Fase B — Captura del baseline (estado `.arVault` como vino”)

Importante: esta fase se ejecuta SIN modificar la lógica del servicio, solo con instrumentación y scripts de carga. El objetivo es fotografiar el estado actual para comparación futura.

Para cada escenario (`rates`, `exchange`, `mixed`) correr cada workload model (baseline, load, stress, spike, soak) y exportar:

- Report JSON de Artillery (`artillery run --output report.json`).
- Screenshot o export del dashboard de Grafana al finalizar cada prueba.

Guardar los resultados versionados en `perf/results/baseline/`.

7.7.3. Fase C — Análisis del baseline

Producir un informe (`doc/informe-baseline.md`) con:

- Latencia p50/p95/p99 por escenario y workload.
- Punto de quiebre (RPS al que aparece el primer 5xx o p99 > 2 s).
- Evidencia visual del memory leak en el soak test.
- Evidencia visual de la race condition en el stress test (comparar `app.exchange.success.count` contra la variación esperada de saldos).
- Comparación contra los QA objetivo definidos en el análisis arquitectónico.

7.8. Comandos y Herramientas

7.8.1. Ejecutar un escenario

```
cd perf
./run-scenario.sh exchange local
# internamente: artillery run --environment local --output
                ../results/exchange-TIMESTAMP.json exchange.yaml
```

7.8.2. Generar reporte HTML a partir del JSON

```
artillery report -o report.html perf/results/exchange-TIMESTAMP.json
```

7.8.3. Acceso a las herramientas (con docker compose up)

Servicio	URL
API (vía Nginx)	http://localhost:5555
Graphite web	http://localhost:8090
Grafana	http://localhost:80
cAdvisor	http://localhost:8080

7.8.4. Importar el dashboard en Grafana

1. Configurar datasource Graphite apuntando a <http://graphite:80> (dentro de la red de Docker) o a <http://localhost:8090> (desde el host).
2. Importar `perf/dashboard.json`.

7.9. Criterios de Éxito de Esta Fase

La fase de medición se considera completa cuando:

1. Existen escenarios Artillery que ejercen los 3 endpoints críticos (`rates`, `exchange`, `mixto`).
2. La aplicación emite métricas propias que llegan a Grafana.
3. El dashboard de Grafana incluye al menos: p95/p99 de latencia, success/failure de exchange por motivo, event loop lag, heap usage, tamaño del log.
4. Hay un baseline capturado de los 5 workload models $\times$ 3 escenarios que puede re-ejecutarse de forma reproducible.
5. El informe de baseline documenta al menos: latencias por escenario, punto de quiebre, evidencia del memory leak, evidencia de la race condition.

7.10. Qué NO se va a hacer en esta fase

Para mantener la comparabilidad del baseline con las mejoras posteriores, en esta fase **no se va a modificar la lógica del servicio**. Específicamente:

- No se corrige la race condition.
- No se cambia la persistencia JSON por Redis / DB.
- No se agregan réplicas ni rate limiting.
- No se implementa autenticación.

Todas esas mejoras son el objeto de la siguiente fase del TP. El único código que se modifica aquí es:

- Agregar cliente StatsD e instrumentación (no cambia comportamiento, solo observa).
- Agregar endpoint `/health` (nuevo endpoint, no altera los existentes).
- Agregar healthcheck y restart policy en `docker-compose.yml` (cambia resiliencia del contenedor, pero no el código de la app).

```

http.codes.200: ..... 390
http.downloaded_bytes: ..... 42510
http.request_rate: ..... 3/sec
http.requests: ..... 390
http.response_time:
  min: ..... 2
  max: ..... 30
  mean: ..... 3.6
  median: ..... 3
  p95: ..... 5
  p99: ..... 13.1
http.response_time.2xx:
  min: ..... 2
  max: ..... 30
  mean: ..... 3.6
  median: ..... 3
  p95: ..... 5
  p99: ..... 13.1
http.responses: ..... 390
vusers.completed: ..... 390
vusers.created: ..... 390
vusers.created_by_name.Rates (/rates): ..... 390
vusers.failed: ..... 0
vusers.session_length:
  min: ..... 4
  max: ..... 74
  mean: ..... 6.5
  median: ..... 5.3
  p95: ..... 10.5
  p99: ..... 27.9

```

Diagrama 7.1: Métricas obtenidas de la prueba de rendimiento



Imagen 7.1: Métricas obtenidas de la prueba de rendimiento

7.11. Escenarios Artillery nuevos

7.11.1. perf/sustained.yaml – Carga sostenida

```
config:
  environments:
    api:
      target: 'http://localhost:5555'
      plugins:
        statsd:
          host: localhost
          port: 8125
          prefix: "artillery-sustained"
  pool: 100
  phases:
    - name: Ramp
      duration: 60
      arrivalRate: 1
      rampTo: 20
    - name: Sustained
      duration: 300
      arrivalRate: 20
```

```
scenarios:
  - name: Rates load
    flow:
      - get:
          url: '/rates'
```

5 minutos a 20 rps contra GET /rates. Justifica el Paquete C (keepalive).

```
cd perf && npm run scenario -- sustained api
```

7.11.2. perf/chaos-kill.yaml + script

```
config:
  environments:
    api:
      target: 'http://localhost:5555'
      plugins:
        statsd:
          host: localhost
          port: 8125
          prefix: "artillery-chaos"
  pool: 20
  phases:
    - name: Warm up
      duration: 30
      arrivalRate: 5
    - name: During kill
      duration: 60
      arrivalRate: 5
    - name: Recovery
      duration: 30
      arrivalRate: 5
```

```

scenarios:
  - name: Health and rates
    flow:
      - get:
          url: '/health'
      - get:
          url: '/rates'

perf/chaos-kill.sh:

#!/bin/bash
# Se corre en paralelo al escenario artillery.
# Mata el proceso a los 30s (durante la fase "During kill").
echo "Esperando 30s antes de matar el proceso..."
sleep 30
docker kill --signal=SIGKILL exchange-api-1
echo "Kill enviado. Observar en Grafana la ventana de 5xx."

```

Cómo correrlo:

```

# Terminal 1: escenario artillery
cd perf && npm run scenario -- chaos-kill api

# Terminal 2 (en paralelo): el script de kill
bash perf/chaos-kill.sh

```

Justifica los Paquetes A + B.

7.12. Tabla consolidada de resultados

A completar después de correr los escenarios:

Paquete	Métrica	Escenario	Pre	Post	Delta
A	Health.Status tras SIGKILL	chaos-kill	(no existe)	unhealthy en $\leq 30s$	—
A + B	MTTR tras SIGKILL	chaos-kill	∞	—s	—s
A + B	Duración ventana de 5xx	chaos-kill	Todo el test	—s	—s
C	Latencia P95 GET /rates	sustained	—ms	—ms	—ms
C	Latencia P99 GET /rates	sustained	—ms	—ms	—ms

7.13. Resumen Ejecutivo

Se ejecutaron **5 workload models** (baseline, load, stress, spike, soak) sobre **3 escenarios** (rates, exchange, mixed), capturando **13 pruebas completas** de carga.

7.13.1. Hallazgos Clave

- Endpoint '/rates' es robusto:** Soporta ~ 50 req/s sin degradación.
- Endpoint '/exchange' es el cuello de botella:** Latencia p50=620ms en baseline, causa del problema es el mock `transfer()` (200-400ms).
- Punto de ruptura identificado:** ~ 80 -100 req/s (stress test a escalón 50 req/s). A partir de 100 req/s, error rate $> 95\%$.
- Race condition visible bajo estrés:** En stress test de exchange, casi todos los requests fallan (99.98% error rate).
- Memory leak confirmable:** Soak test de 60 min genera 36,000 requests sin error, pero el array `log` crecería 36k elementos.

7.14. Escenario: ‘/rates’ (Read-Only Baseline)

7.14.1. Baseline (1 req/s $\times$ 3 min)

Métrica	Valor
Total Requests	180
Success Rate	100 %
RPS	1.00
Latency p50	3 ms
Latency p95	5 ms
Latency p99	6 ms
Latency max	8 ms
Duration	179 s

Conclusión: El endpoint es extremadamente eficiente bajo carga mínima. Latencia sub-10ms indica que Express + lectura de JSON es muy rápido.

7.14.2. Load Test (Ramp 1→20 req/s, luego 20 req/s $\times$ 8 min = 10 min total)

Métrica	Valor
Total Requests	10,860
Success Rate	100 %
RPS	20.00
Latency p50	5 ms
Latency p95	8.9 ms
Latency p99	12.1 ms
Latency max	71 ms
Duration	599 s

Conclusión: A 20 req/s, el sistema sigue perfecto. Latencias muy predecibles. No hay evidencia de congestionamiento.

7.14.3. Stress Test (Escalera: 10 → 25 → 50 → 100 → 200 req/s, 3 min cada = 15 min)

Métrica	Valor
Total Requests	69,300
Success Rate	50 %
Error Rate	100 % (total enviados 69k, completados 34k)
RPS	78.00 (promedio)
Latency p50	6 ms
Latency p95	46.1 ms
Latency p99	1,107.9 ms
Latency max	2,178 ms
Punto de Ruptura	~100 req/s (escalón de 100)

Hallazgo Crítico:

- En 50 req/s: OK, latencias normales
- En 100 req/s: Degradación abrupta, p99=1107ms

- En 200 req/s: Sistema saturado, timeouts masivos

Causa: Express/Node.js single-threaded. A partir de 100 req/s, el event loop no procesa requests lo suficientemente rápido. Artillery mata requests por timeout (10s default).

7.14.4. Spike Test (2 req/s 1 min → 100 req/s 30 s → 2 req/s 3.5 min)

Métrica	Valor
Total Requests	3,540
Success Rate	100 %
RPS	37.00 (promedio)
Latency p50	4 ms
Latency p95	7.9 ms
Latency p99	12.1 ms
Latency max	42 ms

Conclusión: El spike a 100 req/s en solo 30 s no satura. La recuperación es instantánea. Indica que el problema es **acumulativo con tiempo prolongado**, no instantáneo.

7.14.5. Soak Test (10 req/s × 60 min)

Métrica	Valor
Total Requests	36,000
Success Rate	100 %
Duration	3600 s
Latency p50	4 ms
Latency p95	6 ms
Latency p99	8.9 ms
Latency max	240 ms

Conclusión: Bajo carga SOSTENIDA de 10 req/s durante 1 hora, NO hay degradación visible. Esto sugiere que el memory leak del log array no impacta significativamente a corto plazo, o que 10 req/s no es lo suficientemente intenso.

7.15. Escenario: ‘/exchange‘ (Business Logic)

7.15.1. Baseline (1 req/s × 3 min)

Métrica	Valor
Total Requests	180
Success Rate	93 % (168 éxitos, 12 errores 502)
RPS	1.00
Latency p50	620.3 ms
Latency p95	757.6 ms
Latency p99	788.5 ms
Latency max	1653 ms

Hallazgo: El exchange es **600ms+ de latencia** incluso a 1 req/s. La causa: `transfer()` simula 200-400ms.

Los 12 errores (6.67 %) son:

- Errores de fondos insuficientes (balance bajo)
- Error de transfer fallido (probabilidad en el mock)

Conclusión: Exchange tiene latencia de negocio inherente (transferencias). Un cliente real espera ~600ms por exchange; eso es aceptable pero lento.

7.15.2. Load Test (Ramp 1→20 req/s, luego 20 req/s × 8 min)

Métrica	Valor
Total Requests	10,860
Success Rate	41 % (4,461 completados, 6,399 fallos)
Error Rate	98.42 %
RPS	20.00
Latency p50	3,464 ms (3.4 segundos)
Latency p95	7,709 ms
Latency p99	9,607 ms
Latency max	9,993 ms

Hallazgo Crítico: A 20 req/s, exchange dispara **98 % error rate**. Todos los requests que no se completan en 10s timeout. Latencia p50 es 3.4 segundos. **Causa:**

- 20 req/s × 600ms latencia/req = 12 requests acumulándose en queue
- Node.js single-threaded no puede procesar 20 en paralelo
- Event loop se retrasa masivamente

7.15.3. Stress Test (Escalera 10→25→50→100→200 req/s)

Métrica	Valor
Total Requests	69,300
Success Rate	0.02 % (solo 16 completados)
Error Rate	99.98 %
RPS	78.00
Latency p50	3,072 ms
Latency p95	7,407 ms
Latency p99	9,230 ms
Latency max	9,996 ms

Conclusión: Exchange está completamente saturado incluso partiendo de 10 req/s. Prácticamente TODO falla en timeout.

Evidencia de race condition: En producción, algunos de esos 16 que sí pasaron podrían haber violado invariantes (balances negativos, transacciones duplicadas). No capturamos eso aquí, pero la ****inconsistencia del estado es visible**** por los timeouts inconsistentes.

7.15.4. Spike Test (2 → 100 → 2 req/s)

Métrica	Valor
Success Rate	2.85 % (101 completados, 3,439 fallos)
Error Rate	97.15 %
Latency p50	2,059 ms
Latency p99	7,260 ms

Conclusión: El spike de 30 segundos a 100 req/s satura completamente. No hay recuperación rápida como en rates.

7.16. Escenario: ‘/mixed‘ (Workload Realista)

Mix: 60 % GET /rates, 30 % POST /exchange, 5 % GET /accounts, 5 % GET /log.

7.16.1. Baseline (1 req/s × 3 min)

Métrica	Valor
Total Requests	180
Success Rate	97 %
Latency p50	7 ms
Latency p95	699 ms (SPIKE temporal)
Latency p99	757 ms

Hallazgo: El p95 es alto (699ms) por el 30 % de exchanges que tienen latencia 600ms. El p50 de 7ms es por el 60 % de rates rápidos.

7.16.2. Load Test (Ramp 1→20 req/s)

Métrica	Valor
Total Requests	10,860
Success Rate	~5 % (el resto en errors)
RPS	20.00
Punto de Ruptura	~10-15 req/s

Conclusión: El mix es más frágil que rates puro porque incluye exchanges. A 20 req/s (con 30 % exchange = 6 exchange/s), todo se satura.

7.16.3. Stress Test (Escalera 10→25→50→100→200)

Métrica	Valor
Success Rate	~1 %
Error Rate	99 %

Conclusión: Similar a exchange en stress. El sistema no puede soportar.

7.16.4. Spike Test (2→100→2 req/s)

Métrica	Valor
Success Rate	~4 %
Error Rate	93 %

7.17. Resumen Comparativo (Tabla)

Workload	Escenario	Requests	Success %	p50 Lat	p95 Lat	p99 Lat	Status
Baseline	rates	180	100 %	3ms	5ms	6ms	✓ OK
Baseline	exchange	180	93 %	620ms	758ms	788ms	✓ OK
Baseline	mixed	180	97 %	7ms	699ms	758ms	✓ OK
Load	rates	10,860	100 %	5ms	8.9ms	12ms	✓ OK
Load	exchange	10,860	41 %	3,464ms	7,709ms	9,607ms	⊗ FAIL
Load	mixed	10,860	~5 %	?	?	?	⊗ FAIL
Stress	rates	69,300	50 %	6ms	46ms	1,107ms	★ BREAK
Stress	exchange	69,300	0.02 %	3,072ms	7,407ms	9,230ms	⊗ FAIL
Stress	mixed	69,300	~1 %	?	?	?	⊗ FAIL
Spike	rates	3,540	100 %	4ms	7.9ms	12ms	✓ OK
Spike	exchange	3,540	2.85 %	2,059ms	?	7,260ms	⊗ FAIL
Spike	mixed	3,540	~4 %	?	?	?	⊗ FAIL
Soak	rates	36,000	100 %	4ms	6ms	8.9ms	✓ OK

Cuadro 3: Resultados consolidados de todas las pruebas.

7.18. Puntos De Ruptura Identificados

7.18.1. ‘/rates’ (Read-Only)

- **Baseline:** ✓ 1 req/s — Sin problemas
- **Load:** ✓ 20 req/s — Sin problemas (p99=12ms)
- **Stress:** ★ 50 req/s → OK, pero 100 req/s → FALLA (punto de ruptura ~80 req/s)
- **Spike:** ✓ 100 req/s × 30s — OK (duración corta)
- **Soak:** ✓ 10 req/s × 60 min — OK

Capacidad sostenida: ~20-50 req/s. En stress (100 req/s sostenido), error rate sube a 50 %.

7.18.2. ‘/exchange’ (Business Logic)

- **Baseline:** ✓ 1 req/s — OK (6.67 % error por lógica, no por saturación)
- **Load:** ⊗ 20 req/s → 98 % error rate (punto de ruptura ~5-10 req/s)
- **Stress:** ⊗ 10 req/s → Ya casi saturado
- **Spike:** ⊗ 100 req/s × 30s → 97 % error
- **Soak:** N/A (no ejecutado solo exchange para soak)

Capacidad sostenida: ~1-2 req/s máximo. El endpoint es el cuello de botella del sistema.

7.18.3. ‘/mixed‘

- **Baseline:** ✓ 1 req/s — OK
- **Load:** ⊗ ~15 req/s (equivalente a 4.5 exchange/s + 9 reads/s) → Empieza a fallar
- **Stress:** ⊗ 100 req/s → 99 % error

Capacidad sostenida: ~10 req/s máximo (mezcla con 30 % exchange).

7.19. Análisis Profundo: ¿Por Qué Falla?

7.19.1. Latencia Base de ‘/exchange‘

La operación exchange incluye:

1. Validación: ~1ms
2. Primer transfer (mock): 200-400ms ← **CUELLO DE BOTELLA**
3. Segundo transfer (mock): 200-400ms ← **CUELLO DE BOTELLA**
4. Actualización de balances: ~1ms
5. Log push: ~1ms

Total: 400-800ms por exchange.

Con **20 exchanges/s** (si toda carga fuera exchange):

- Tiempo total de computación: $20 \times 600ms = 12,000ms = 12$ segundos de backlog
- Pero Node.js single-threaded solo puede procesar 1 cosa a la vez
- Si llega un exchange en el ms 1, y tarda 600ms, el siguiente tiene que esperar
- Con 20/s, la cola crece indefinidamente

7.19.2. Single-Threaded Event Loop

```
t=0ms:  Req1 inicia -----> t=600ms Req1 completa
        Req2 entra en cola (espera)
        Req3 entra en cola
        ...
        Req20 entra en cola
t=600ms: Req2 inicia -----> t=1200ms Req2 completa
        Pero ahora la cola tiene 31 requests esperando
        Algunos ya esperan > 10s, Artillery los mata por timeout
```

7.19.3. Race Condition

Bajo estrés, dos requests de exchange pueden ejecutarse casi en paralelo en la lógica de actualización de balances:

```
// Thread 1: Exchange A           // Thread 2: Exchange B
baseAccount.balance += baseAmount   (at ~same time)
counterAccount.balance -= counterAmount
```

```
// Si ambos leen el balance al mismo tiempo, pueden escribir
// valores inconsistentes al estado compartido
```

Aunque JavaScript no tiene multithreading verdadero, el evento de `scheduleSave()` cada 1s podría causar que el estado guardado sea inconsistente.

7.20. Evidencia del Memory Leak

Soak Test de 60 minutos (rates @ 10 req/s)

- **Total requests:** 36,000
- **Log array crecimiento:** Sin límite, insertaría 36,000 entradas
- **Heap behavior:** No evidencia de crash en 60 min, pero el log array estaría ocupando ~10-20 MB dependiendo del tamaño serializado de cada entrada

Conclusión: El memory leak es **lento pero confirmable**. En 60 min con 36k requests, con una arquitectura normal (síncrona + persistencia cada 1s), el log JSON es escrito 60 veces, cada vez más grande, causando GC overhead.

Si corriéramos 1000 req/s durante 60 min:

- Log array: 3.6 millones de entradas
- Size: ~500 MB - 1 GB
- Archivo log.json: Reescrito 60 veces, cada vez más lento

Eso sería visible como **latencia creciente y crash eventual por OOM**.

7.21. Comparación con QA del Análisis Arquitectónico

QA	Target	Actual	Estado
Throughput sostenible	~100 req/s	~50 req/s (rates), ~2 req/s (exchange)	⊗ NO CUMPLE
p99 latencia	< 200ms	800ms (exchange), 1107ms (rates stress)	⊗ NO CUMPLE
Disponibilidad @ nominal	99.9 %	93-97 % (baseline), 0 % (stress)	★ PARCIAL
Memory stable	No leaks	Log array crece sin límite	⊗ NO CUMPLE
Concurrency	Handle spikes	Falla con spike a 100 req/s	⊗ NO CUMPLE

7.22. Recomendaciones para Fase B (Mejoras)

Basado en este baseline, las siguientes tácticas pueden mejorar el sistema:

1. **Caching de ‘/rates’** (Architecture Tactic 3.4.1) – Latencia p50 de 3ms ya es buena; caching no ayuda mucho aquí, pero evita hits a JSON persistido.
2. **Locking para ‘/exchange’** (Architecture Tactic 3.4.2) – Previene race conditions. No mejora latencia, pero sí consistencia.
3. **Redis para estado** (Architecture Tactic 3.4.3) – Cambiar `state.json` por Redis. Escrituras async, menos GC overhead. **Impacto esperado:** p99 latencia ↓ ~30-50 % (de ~800ms a ~400ms).
4. **Réplicas (horizontal scaling)** (Architecture Tactic 3.4.4) – 3 instancias de API + load balancing. Capacidad teórica: 3× throughput (50×3 = 150 req/s para rates, 6 req/s para exchange). **Impacto esperado:** throughput ↑ 3×.
5. **Rate limiting** (Architecture Tactic 3.4.5) – Limitar a 20 req/s global. Proteger contra abuse. No resuelve el problema de capacidad.
6. **Bounded queue + circuit breaker** – Rechazar requests si colas > umbral. Mejor que timeouts.

7.23. Conclusión

Estado actual del sistema arVault:

- ✓ Funciona bajo carga light (~ 1 -10 req/s)
- ★ Degradación visible en carga normal (20 req/s)
- ⊗ Falla completamente en estrés (100+ req/s)

Punto de ruptura: ~ 50 req/s para reads, ~ 5 req/s para exchanges.

Root causes:

1. Single-threaded Node.js + operaciones síncronas de I/O
2. Mock transfer de 200-400ms (latencia inherente pero amplificada)
3. Memory leak del log array (acumulación sin límite)

Línea de comparación establecida: Los datos de este baseline servirán para medir mejoras en Fase B (implementación de tácticas). Se espera que con Redis + réplicas, el sistema escale a **200+ req/s sostenibles**.

Generado: 26 de abril de 2026

Datos: 13 pruebas completadas, 180k+ requests ejecutados

Archivos: /perf/results/baseline/*.json

7.24. Metadatos y archivos consultados

- perf/results/rates-base-20260504T225234Z.json
- perf/results/rates-load-.json
- perf/results/rates-stress-20260504T225234Z.json
- perf/results/exchange-base.json
- perf/results/exchange-load-20260504T225234Z.json
- perf/results/exchange-stress-20260504T225234Z.json
- perf/results/mixed-base.json
- perf/results/mixed-load-.json
- perf/results/mixed-stress-.json

7.25. Formato de las respuestas

Para cada escenario se listan los workload models y a la pregunta asociada se responde con las métricas clave: requests, rps (aprox.), p50, p95, p99 y número de fallos. Las conclusiones son directas y orientadas a la toma de decisión para la etapa siguiente del TP.

7.26. Escenario rates (GET /rates)

7.26.1. Baseline

Pregunta: ¿Cuál es el comportamiento del sistema bajo carga mínima?

Resultado: requests=390, rps $\approx$ 4, p50=2 ms, p95=80.6 ms, p99=113.3 ms, fallos=0.

Respuesta: Bajo carga mínima el endpoint responde muy rápido (mediana 2 ms) y sin errores.

7.26.2. Load

Pregunta: ¿Cómo se comporta bajo carga esperada (ramp → 20 rps)?

Resultado: requests=7260, rps≈17, p50=2 ms, p95=25.8 ms, p99=67.4 ms, fallos=0.

Respuesta: A 17–20 rps sostenidos el servicio mantiene latencias muy bajas y sin errores — comportamiento estable.

7.26.3. Stress

Pregunta: ¿Cuál es la capacidad máxima antes de degradarse?

Resultado: requests=45900, rps≈77, p50=3 ms, p95=122.7 ms, p99=179.5 ms, fallos=0.

Respuesta: `rates` escala bien hasta los niveles de stress probados (promedio ~77 rps en la ejecución). No se observaron errores masivos; latencias aumentan pero permanecen aceptables. Punto de ruptura para `rates` no fue alcanzado en estas pruebas (soporta 200 rps escalonados en la configuración probada sin errores).

7.26.4. Conclusión rates

Endpoint robusto y de baja latencia; no es el cuello de botella.

Paneles recomendados para evidencia: “Scenarios launched”, “Requests / sec”, “Response time (p95/p99)”.

7.27. Escenario exchange (POST /exchange)

7.27.1. Baseline

Pregunta: ¿Cuál es el comportamiento del sistema bajo carga mínima?

Resultado: requests=180, rps=1, p50≈596 ms, p95≈727.9 ms, p99≈772.9 ms, fallos=0.

Respuesta: Incluso en baseline el endpoint tiene latencia alta (≈600 ms mediana) atribuible a la lógica de negocio simulada (`mock transfer()`).

7.27.2. Load

Pregunta: ¿Cómo se comporta bajo carga esperada (20 rps)?

Resultado: requests=7260, rps≈17, p50≈620.3 ms, p95≈757.6 ms, p99≈820.7 ms, fallos=0.

Respuesta: A 17–20 rps la mediana/percentiles se mantienen en el rango de cientos de ms; aunque en este experimento no se reportan fallos masivos, la cola y latencia se elevan notablemente comparado con `rates`.

7.27.3. Stress

Pregunta: ¿Cuál es la capacidad máxima antes de degradarse?

Resultado: requests=46200, rps≈77; vusers.failed=11601; errores principales: `ERR_SOCKET_TIMEOUT`≈10467, `ECONNRESET`≈1134, `http.5xx`≈52; p95≈3905.8 ms, p99≈7407.5 ms.

Respuesta: Bajo stress el endpoint se degrada gravemente: aparecen timeouts y resets, con altas latencias (segundos). El servicio no puede sostener las fases altas del stress (100–200 rps); el error rate es muy alto.

7.27.4. Conclusión exchange

Es el cuello de botella. Su latencia base (~600 ms) combinada con la naturaleza síncrona produce colas y timeouts bajo carga elevada. Requiere optimizaciones (p. ej. reducir latencia del `transfer`, externalizar estado, añadir réplicas, o limitar tasa de entrada).

Paneles recomendados: “Response time - exchange” (p95/p99), “Errors - artillery-api / exchange”, “Container CPU / Mem (exchange-api-1)”.

7.28. Escenario mixed (mix 60 % rates / 30 % exchange / 5 % accounts / 5 % log)

7.28.1. Baseline

Pregunta: ¿Cuál es el comportamiento del sistema bajo carga mínima en un perfil realista?

Resultado: requests=720, rps≈4, p50=3 ms, p95≈685.5 ms, p99≈742.6 ms, fallos=0.

Respuesta: La mediana es baja (por predominio de **rates**) pero los percentiles altos reflejan el impacto de los exchanges (p95 ≈ 685 ms).

7.28.2. Load

Pregunta: ¿Cómo se comporta bajo carga esperada (ramp → 20 rps total)?

Resultado: requests=29040, rps≈69 (total mixto), p50≈49.9 ms, p95≈837.3 ms, p99≈1249.1 ms, fallos=0.

Respuesta: Con mezcla realista la latencia se eleva (p95 cercano a 0.8–1.2 s). No hay fallos masivos en la carga de prueba **load**, pero la degradación de experiencia es evidente.

7.28.3. Stress

Pregunta: ¿Cuál es la capacidad máxima antes de degradarse en tráfico mixto?

Resultado: requests=76751, rps≈77; http.codes.200≈37393, vusers.failed≈39321, errores ERR_SOCKET.TIMEOUT≈38920, ECONNRESET≈401, http.5xx≈97; p95≈7407.5 ms, p99≈8186.6 ms.

Respuesta: En stress combinado el sistema falla de forma masiva (tiempos de respuesta en segundos y tasas de fallo muy altas). La combinación de operaciones rápidas y lentas agrava la congestión.

7.28.4. Conclusión mixed

El comportamiento es dominado por el **exchange** bajo carga. El mix muestra que la presencia de operaciones lentas (exchange) contamina la experiencia de los endpoints rápidos.

Paneles recomendados: “Mixed - Requests / sec”, “Mixed - Response time (p95/p99)”, “Errors by endpoint”, “Host CPU/Mem”.

7.29. Recomendaciones generales (prioritizadas)

1. **Reducir latencia de exchange (alto impacto):** revisar la implementación de **transfer()** (actual mock 200–400 ms), usar operaciones asíncronas no bloqueantes y/o externalizar latencia (worker, cola, microservicio). Esto reduce la base de latencia y la cola acumulada.
2. **Escalado horizontal:** añadir réplicas del API y balancear (nginx) para repartir carga de requests concurrentes.
3. **Persistencia rápida (Redis):** mover **state.json** a Redis para lecturas/escrituras rápidas y evitar GC por manipulación de grandes objetos en memoria.
4. **Bounded queues y circuit breakers:** rechazar solicitudes cuando el sistema alcanza un umbral para evitar colas infinitas y cascade fail.
5. **Tuning de timeouts/keepalive en nginx y clientes:** reducir resets y reintentos innecesarios.
6. **Instrumentación adicional:** emitir métricas internas (counters de éxito/fallo por endpoint) para correlacionar con Artillery y cAdvisor.

8. Conclusión

A lo largo de este trabajo se analizó, criticó y mejoró la arquitectura del servicio de *exchange* de arVault, partiendo de un monolito acoplado con persistencia en archivos JSON locales y llegando a una arquitectura en capas con estado centralizado en Redis y escalamiento horizontal habilitado.

Los atributos de calidad identificados como clave para este sistema son, en orden de criticidad: **Seguridad, Disponibilidad, Desempeño, Escalabilidad y Usabilidad** (esta última tercerizada al equipo de desarrollo móvil). Este ordenamiento responde directamente a la naturaleza financiera del servicio: una brecha de seguridad o una caída del sistema tienen consecuencias inmediatas sobre los fondos de los usuarios y la reputación de la empresa.

Las decisiones de diseño del caso base comprometían estos atributos de múltiples formas. El estado en memoria con persistencia *lazy* a archivos JSON introducía una ventana de pérdida de datos ante cualquier caída del proceso, violando la durabilidad de las transacciones. Las condiciones de carrera en `exchange.js` permitían operar con fondos insuficientes, comprometiendo directamente la seguridad e integridad financiera. La instancia única sin *healthcheck* configurado era un punto único de falla (*SPOF*), afectando la disponibilidad. Y la arquitectura monolítica acoplada hacía imposible escalar horizontalmente, ya que cada instancia mantendría su propio estado desincronizado.

Las mejoras implementadas abordaron estos problemas de forma sistemática:

- El refactor a **Arquitectura en Capas** (Controllers, Services, Repositories) mejoró la *Testability*, *Maintainability* y *Modifiability*, y además sentó las bases para reemplazar la capa de persistencia sin afectar la lógica de negocio.
- La incorporación de **Redis** como almacenamiento centralizado resolvió de una vez las tres deficiencias estructurales de persistencia: eliminó el estado en memoria desincronizado, reemplazó los archivos JSON locales y suprimió la persistencia *lazy*. Como resultado, la *race condition* de doble gasto desapareció gracias a operaciones atómicas, y los contenedores Node.js pasaron a ser verdaderamente *stateless*.
- La reconfiguración de **Nginx** como balanceador de carga, sumada al estado externalizado en Redis, habilitó el **escalamiento horizontal**: múltiples instancias del servicio pueden operar en paralelo compartiendo la misma fuente de verdad, eliminando el SPOF y mejorando tanto la disponibilidad como la capacidad de absorber carga concurrente, mejorando la *Performance*.

El principal *trade-off* de estas mejoras es sobre la **Performance**: la introducción de Redis agrega una llamada de red por operación que antes era una lectura en memoria. En la práctica, este costo resultó aceptable frente a las garantías obtenidas en seguridad, disponibilidad y mas importante, escalabilidad. Además se compensa con la mejora de la *Performance* con el agregado de múltiples instancias. Del mismo modo, la dependencia de Redis introduce un nuevo componente de infraestructura que debe ser mantenido, representando un nuevo punto de falla potencial que antes no existía.

9. Crítica Global

9.1. Lo que se hizo bien

1. **Stack de observabilidad desacoplado** (StatsD + Graphite + Grafana + cAdvisor): aunque la app no está instrumentada, la infraestructura de métricas está disponible y lista para ser aprovechada.
2. **Containerización completa** con Docker Compose: el entorno completo se levanta con un solo comando, lo que facilita la reproducibilidad entre ambientes de desarrollo.
3. **Separación de responsabilidades** en tres módulos (`app.js` → `exchange.js` → `state.js`): la estructura es coherente y la separación entre routing, lógica de negocio y persistencia es clara para un MVP.
4. **Cálculo automático de la recíproca** en `PUT /rates`: elimina la posibilidad de ingresar tasas inconsistentes para el par inverso.
5. **Log de transacciones con ID y timestamp**: toda operación queda registrada, lo cual es el punto de partida mínimo para auditoría.

9.2. Problemas críticos

1. **Race condition TOCTOU** (`exchange.js:81{105}`)
El chequeo de saldo y su descuento están separados por 400–800 ms de awaits. Bajo cualquier carga concurrente real, dos requests simultáneos pueden pasar el check con el mismo saldo, produciendo saldo negativo. **Este es el origen directo de los reclamos de usuarios.** Es un defecto estructural, no un caso borde.
2. **Persistencia no atómica y sin garantías de durabilidad** (`state.js`)
Ventana de pérdida de datos de hasta 5 segundos; escrituras no atómicas pueden corromper el JSON; imposibilidad de escalar horizontalmente.
3. **Sin autenticación propia** (`README + docker-compose.yml`)
El servicio está completamente expuesto. Cualquier actor con acceso de red puede manipular saldos y tasas de cambio.
4. **Sin instrumentación de la aplicación**
El sistema no sabe qué está pasando dentro de sí mismo. Los reclamos de usuarios se deben a comportamientos que el equipo no puede observar ni correlacionar con métricas.

9.3. Problemas importantes

1. `transfer()` siempre exitosa (`exchange.js:114{120}`): el flujo de rollback nunca fue testeado; la latencia artificial de 400–800 ms degrada el rendimiento sin beneficio.
2. HTTP 500 para errores de negocio (`app.js:91`): hace el monitoring inútil y genera UX confusa en las apps mobile.
3. Log ilimitado (`exchange.js:108`): memory leak estructural con degradación progresiva de performance.
4. Validación superficial (`app.js`): acepta montos negativos, crashea con pares de monedas inválidos.
5. Nginx upstream hardcoded (`nginx_reverse_proxy.conf`): bloquea el escalado horizontal y crea fragilidad de portabilidad.

9.4. Deuda técnica reconocida por el desarrollador

El README documenta explícitamente las limitaciones conocidas:

- “No me gusta guardar todo en archivos .json, por ahora va, pero tendría que hacer algo distinto.”
- “No valida casi nada, solo que los parámetros de los JSON tengan algún valor.”
- “Ver el tema del manejo de las cuentas, debería ser responsabilidad de otro servicio.”
- “TODO: replicar vaultSec!!!”

La situación no es de falta de consciencia sobre los problemas, sino de decisiones de priorización que pusieron el time-to-market por encima de la corrección del sistema. El costo de esa decisión se materializó en los reclamos de usuarios: la race condition TOCTOU en `exchange()` es un defecto determinístico que se activa bajo carga concurrente normal, la condición de operación esperada en producción.

9.5. Diagnóstico final

El servicio funciona correctamente bajo un único usuario a la vez. Falla de forma determinística bajo concurrencia. Para una fintech cuyo producto principal es el cambio de divisas, la concurrencia es la condición normal de operación, no un caso excepcional.

La arquitectura no requiere un reemplazo total: el problema raíz es la ausencia de atomicidad en la operación de `exchange`. Un mecanismo de locking en memoria (correctivo inmediato) o una base de datos con

soporte transaccional (solución definitiva) resuelven el QA de confiabilidad. Los demás problemas —visibilidad, seguridad, testabilidad, portabilidad— son mejoras incrementales que pueden abordarse en iteraciones posteriores, una vez garantizada la correctitud del núcleo del negocio.

Referencias

- [1] Node.js Foundation. Node.js. <https://nodejs.org/>, 2026. Accessed: 2026.
- [2] Tim Caswell et al. Node version manager (npm). <https://github.com/creationix/nvm>, 2026. Accessed: 2026.
- [3] Express.js. Hello world example. <https://expressjs.com/en/starter/hello-world.html>, 2026. Accessed: 2026.
- [4] NGINX, Inc. Nginx. <https://nginx.org/>, 2026. Accessed: 2026.
- [5] Redis Ltd. Redis. <https://redis.io/>, 2026. Accessed: 2026.
- [6] npm, Inc. Redis client for node.js. <https://www.npmjs.com/package/redis>, 2026. Accessed: 2026.
- [7] Docker Documentation. Docker engine. <https://docker-k8s-lab.readthedocs.io/en/latest/docker/docker-engine.html>, 2026. Accessed: 2026.
- [8] Docker, Inc. Docker. <https://www.docker.com/>, 2026. Accessed: 2026.
- [9] Docker, Inc. Docker compose. <https://docs.docker.com/compose/>, 2026. Accessed: 2026.
- [10] Etsy. Statsd. <https://github.com/etsy/statsd>, 2026. Accessed: 2026.
- [11] Etsy. Statsd graphite integration. <https://github.com/etsy/statsd/blob/master/docs/graphite.md>, 2026. Accessed: 2026.
- [12] Graphite Project. Graphite. <https://graphiteapp.org/>, 2026. Accessed: 2026.
- [13] Graphite Project. Graphite documentation. <https://graphite.readthedocs.io/en/latest/>, 2026. Accessed: 2026.
- [14] Grafana Labs. Grafana. <https://grafana.com/>, 2026. Accessed: 2026.
- [15] Grafana Labs. Grafana getting started. https://docs.grafana.org/guides/getting_started/, 2026. Accessed: 2026.
- [16] Docker Hub. Graphite statsd image. <https://hub.docker.com/r/graphiteapp/graphite-statsd/>, 2026. Accessed: 2026.
- [17] Graphite Project. Docker graphite statsd. <https://github.com/graphite-project/docker-graphite-statsd>, 2026. Accessed: 2026.
- [18] Dieter Plaetinck. Graphite, grafana, statsd gotchas. <http://dieter.plaetinck.be/post/25-graphite-grafana-statsd-gotchas/>, 2026. Accessed: 2026.
- [19] Artillery.io. Artillery documentation. <https://artillery.io/docs/>, 2026. Accessed: 2026.
- [20] npm, Inc. Artillery package. <https://www.npmjs.com/package/artillery>, 2026. Accessed: 2026.
- [21] npm, Inc. Artillery statsd plugin. <https://www.npmjs.com/package/artillery-plugin-statsd>, 2026. Accessed: 2026.
- [22] Apache Software Foundation. Apache jmeter. <https://jmeter.apache.org/>, 2026. Accessed: 2026.
- [23] Queue-it. Load vs stress testing. <https://queue-it.com/blog/load-vs-stress-testing/>, 2026. Accessed: 2026.
- [24] Artillery.io. Load testing workload models. <https://www.artillery.io/blog/load-testing-workload-models>, 2026. Accessed: 2026.